B.Tech. BCSE497J - Project-I

ADAPTIVE PORTFOLIO OPTIMIZATION IN DYNAMIC FINANCIAL MARKETS

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering (Artificial Intelligence and Machine Learning)

by

23BAI0002	**YOGDEEP BENCHIMATH**
23BAI0014	**ANUSHKA MISHRA**
23BAI0044	**MEHUL ARORA**

Under the Supervision of

Dr. Arti Tiwari

[Designation of the guide]

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled **ADAPTIVE PORTFOLIO OPTIMIZATION IN DYNAMIC FINANCIAL MARKETS** submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by us under the supervision of Dr. Arti Tiwari.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature of the Candidates
Yogdeep Benchimath (23BAI0002)
Anushka Mishra (23BAI0014)
Mehul Arora (23BAI0044)

CERTIFICATE

This is to certify that the project entitled **ADAPTIVE PORTFOLIO OPTIMIZATION IN DYNAMIC FINANCIAL MARKETS** submitted by **Yogdeep Benchimath (23BAI0002), Anushka Mishra (23BAI0014) and Mehul Arora (23BAI0044)**, School of Computer Science and Engineering, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute orany other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Dr. Arti Tiwari

Examiner(s)

[Name of the Head of the Department]

B.Tech. Computer Science and Engineering (Artificial Intelligence and Machine Learning)

ACKNOWLEDGEMENTS

We thank the management of Vellore Institute of Technology (VIT) for the opportunity to take up this project and for the academic environment in which we could carry it out.

We are grateful to Dr. Jaisankar N, Dean of the School of Computer Science and Engineering (SCOPE), for the support and encouragement the school has given us during our studies.

We thank [Name of the Head of the Department], Head of the [Department Name], for the guidance and support given to us while we worked on this project.

We are especially thankful to our project guide, Dr. Arti Tiwari, for the guidance, time and feedback that shaped this project.

Thank you all for your contributions and support.

Yogdeep Benchimath
Anushka Mishra
Mehul Arora

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**xii**
1.	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Motivations	1
	1.3 Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	**3**
	2.1 Literature Review	3
	2.2 Research Gap	3
	2.3 Objectives	4
	2.4 Problem Statement	5
	2.5 Project Plan	5
3.	**TECHNICAL SPECIFICATION**	**7**
	3.1 Requirements	7
	3.1.1 Functional	7
	3.1.2 Non-Functional	8
	3.2 Feasibility Study	8
	3.2.1 Technical Feasibility	8
	3.2.2 Economic Feasibility	8
	3.2.3 Social Feasibility	9
	3.3 System Specification	9
	3.3.1 Hardware Specification	9
	3.3.2 Software Specification	10
4.	**DESIGN APPROACH AND DETAILS**	**11**
	4.1 System Architecture	11
	4.2 Design	12
	4.2.1 Data Flow Diagram	12
	4.2.2 Use Case Diagram	13
	4.2.3 Class Diagram	14
	4.2.4 Sequence Diagram	15

5. METHODOLOGY AND TESTING 18

5.1 Module Description 18

5.1.1 Data Acquisition and Preprocessing 18

5.1.2 Regime Detection with a Hidden Markov Model 20

5.1.3 Length-Invariant Shape and Transformer Encoder 20

5.1.4 Reference Corpus and HDBSCAN 21

5.1.5 Similarity Scoring and the Comparison Report 21

5.1.6 Allocation Strategies 22

5.1.7 Backtest Engine and Evaluation Metrics 23

5.1.8 Dashboard 24

5.2 Testing 25

5.2.1 Approach and Test Environment 25

5.2.2 Functional (API) Tests 26

5.2.3 Unit Tests 27

5.2.4 Model Validation Protocol 27

5.2.5 Defects Found and Fixed 28

6. PROJECT DEMONSTRATION 29

6.1 Running the Application 29

6.2 Portfolio Backtest Tab 29

6.3 Regime Detection Tab 30

6.4 About Tab 32

7. RESULT AND DISCUSSION (COST ANALYSIS AS APPLICABLE) 34

7.1 Regime Detection 34

7.2 Historical-Pattern Comparison 34

7.2.1 Example Reports 34

7.2.2 Embedding Space 35

7.2.3 Leakage-Controlled Evaluation 36

7.2.4 Unseen Crises 37

7.3 Walk-Forward Allocation Backtests 37

7.3.1 Criterion-Wise Analysis 40

7.4 Statistical Analysis 41

7.5 Ablation Study 41

7.6 Comparison with Existing Methods 43

7.7 Cost Analysis 43

7.8 Discussion 44

7.9 Limitations 44

8. CONCLUSION 46

8.1 Future Work 46

9. REFERENCES 48

APPENDIX A: SAMPLE CODE 50

List of Figures

Figure No.	Title	Page No.
2.1	Four-month project work plan	6
4.1	Layered architecture of the Adaptive Portfolio Optimizer	11
4.2	Data flow diagram, level 0 (context diagram)	12
4.3	Data flow diagram, level 1	12
4.4	Use case diagram	13
4.5	Class diagram of the back end	14
4.6	Sequence diagram of a historical-pattern comparison	15
4.7	Sequence diagram of a portfolio backtest	16
5.1	Transformer shape encoder and the decoder used only for training	21
5.2	Historical-pattern comparison pipeline	22
5.3	Walk-forward protocol: training on the pre-crisis segment, evaluation through the crisis	23
6.1	Portfolio Backtest tab for the 2008 crisis	30
6.2	Regime Detection tab for SPY, 2019 to 2021, with the comparison loading	31
6.3	Regime Detection tab with the comparison report for the COVID-19 window	32
6.4	About and tech stack tab	33
7.1	HMM regimes on SPY for (a) the dot-com bust, (b) the 2008 crisis and (c) the COVID-19 crash	34
7.2	Query paths (solid) against the nearest reference windows (dashed)	35
7.3	Corpus embeddings colored by label: (a) PCA and (b) t-SNE of the 965 vectors	35
7.4	Embedding ablation under blocked cross-validation	37
7.5	Confusion matrix of the trained-Transformer classifier under blocked cross-validation	37
7.6	Portfolio value of the three strategies in the four settings	39
7.7	Out-of-sample Sharpe ratio, maximum drawdown and volatility	39
7.8	Adaptive-policy weights over time (seed 42)	40
7.9	Out-of-sample Sharpe ratio of the adaptive policy across six seeds; dashed lines are the baselines	42

List of Tables

Table No.	Title	Page No.
1.1	Scope of the project	2
2.1	Literature review of recent adaptive portfolio optimization studies	3
2.2	Research gaps, proposed focus and status	4
2.3	Sustainable Development Goals addressed	5
2.4	Project plan with task durations	6
3.1	Functional requirements	7
3.2	Non-functional requirements	8
3.3	Resources and costs	9
3.4	Hardware used for development and testing	9
3.5	Software stack	10
4.1	REST endpoints of the back end	12
5.1	Backtest settings and crisis definitions	18
5.2	Descriptive statistics of each asset over each backtest window (252 trading days per year)	19
5.3	Hyper-parameters used in the experiments	24
5.4	Functional tests of the REST API	26
5.5	Unit tests of the analytics modules	27
5.6	Defects found during development and testing	28
7.1	Comparison report output for five example queries	34
7.2	Blocked, purged 5-fold cross-validation of event identification	36
7.3	Leave-one-event-out: share of the unseen crisis' windows predicted as crisis-like	37
7.4	Out-of-sample performance of the three strategies (policy seed 42)	37
7.5	Best strategy per criterion and setting on the test segment	40
7.6	Paired stationary-bootstrap intervals for the test-segment Sharpe ratio difference	41
7.7	Allocator ablation: test-segment results, mean ± standard deviation over six seeds	41
7.8	Capability comparison with the reviewed studies	43
7.9	Computing cost of the main operations	44
8.1	Objectives and outcomes	46

List of Abbreviations

AD	Adaptive (strategy)
API	Application Programming Interface
CI	Confidence Interval
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
DDPG	Deep Deterministic Policy Gradient
DFD	Data Flow Diagram
DRL	Deep Reinforcement Learning
EM	Expectation-Maximization
ES	Evolution Strategies
ETF	Exchange-Traded Fund
EW	Equal-Weight (strategy)
FFN	Feed-Forward Network
FR	Functional Requirement
GAT	Graph Attention Network
GELU	Gaussian Error Linear Unit
GFC	Global Financial Crisis
GLD	SPDR Gold Shares (ETF)
GPU	Graphics Processing Unit
HDBSCAN	Hierarchical Density-Based Spatial Clustering of Applications with Noise
HMM	Hidden Markov Model
HTTP	Hypertext Transfer Protocol
IEEE	Institute of Electrical and Electronics Engineers
JSON	JavaScript Object Notation
k-NN	k-Nearest Neighbors
LOEO	Leave-One-Event-Out
LSTM	Long Short-Term Memory
MACD	Moving Average Convergence Divergence
MDD	Maximum Drawdown
MLP	Multi-Layer Perceptron
MV	Mean-Variance (strategy)
NFR	Non-Functional Requirement
PCA	Principal Component Analysis
PPO	Proximal Policy Optimization
QQQ	Invesco QQQ Trust (Nasdaq-100 ETF)
REST	Representational State Transfer
RL	Reinforcement Learning
RSI	Relative Strength Index
SDG	Sustainable Development Goal
SLSQP	Sequential Least Squares Programming

SMA	Simple Moving Average
SPY	SPDR S&P 500 ETF Trust
SR	Sharpe Ratio
t-SNE	t-distributed Stochastic Neighbor Embedding
TLT	iShares 20+ Year Treasury Bond ETF
TO	Turnover
TRL	Technology Readiness Level
UI	User Interface
VIX	CBOE Volatility Index
VUSTX	Vanguard Long-Term Treasury Fund

Symbols and Notations

r_t	Daily log return of an asset on day t
P_t	Adjusted closing price on day t
σ_t	20-day annualized rolling volatility
d_t	Drawdown from the 60-day rolling peak
z_t	Hidden regime state on day t (bull, bear or crisis)
A	State transition matrix of the HMM
μ_j, σ_j	Mean and standard deviation of the features in HMM state j
$\tilde{X}$	Length-invariant 40 by 3 shape of a price window
c_τ, v_τ, g_τ	Cumulative return, rolling volatility and running drawdown at point τ of a window
d_{model}	Hidden size of the Transformer (32)
Q, K, V	Query, key and value matrices of self-attention
d_k	Dimension of the attention keys
f_θ, g_φ	Encoder and decoder of the autoencoder
L	Reconstruction loss
z	16-dimensional embedding of a window
ℓ	Label of a corpus window (dot-com, GFC, COVID-19 or normal)
k	Number of nearest neighbors (8)
s_ℓ	In-class spacing of label ℓ
m_ℓ	Mean distance to the k nearest windows of label ℓ
π_ℓ	Gaussian-kernel similarity to label ℓ
p_ℓ	Local purity of label ℓ among the 15 nearest windows
C_ℓ	Match confidence for label ℓ
w_k	Portfolio weight vector at rebalance k
s_k	State vector given to the adaptive policy
W_1, W_2, b_1, b_2	Weights and biases of the policy network
θ	Flattened parameters of the policy
ε_i	Gaussian perturbation of the parameters in evolution strategies
σ, α	Perturbation scale (0.1) and step size (0.05) of evolution strategies
$\hat{F}_i$	Rank-normalized reward of perturbation i
$R(\theta)$	Training reward of the policy
SR	Sharpe ratio
MDD	Maximum drawdown
TO_k	Turnover at rebalance k
V	Portfolio value
c	Proportional transaction cost (5 basis points)
r_f	Annual risk-free rate (2 percent)
Δ	Difference in Sharpe ratio between two strategies

ABSTRACT

Portfolio managers allocate money with models that assume the market will behave tomorrow as it did in the sample used to fit them. Markets do not behave that way. The dot-com bust, the 2008 financial crisis and the COVID-19 crash each changed volatility, correlation and drawdown patterns within weeks, and static strategies such as mean-variance optimization react slowly. This project asks two practical questions. Does a chosen period of market data resemble one of those three crises, and does an allocation policy that knows the current market regime do better than simple benchmarks?

We built a working web application, with a FastAPI back end and a React dashboard, that answers both questions on real daily prices from Yahoo Finance. A three-state Gaussian hidden Markov model labels each trading day as bull, bear or crisis without being given crash dates. For the comparison, any price window is turned into a 40-point, three-channel shape (cumulative return, short-term volatility and running drawdown), encoded into a 16-dimensional vector by a small Transformer encoder, and scored against a reference corpus of 965 labelled windows from the three crises and from calm periods. HDBSCAN is used to examine how the corpus clusters. The report shows the closest historical event, a confidence value, an overlay with the nearest real window, and a normal-regime baseline when no crisis matches. For allocation, we trained a small neural policy with evolution strategies and compared it with equal-weight and rolling mean-variance portfolios in walk-forward backtests that include a 5 basis point cost.

The system passed 21 functional and unit tests, and the models were evaluated with leakage-controlled validation. The matcher separates crisis-like from normal periods, finding 48.8 percent of crisis windows with a 1.0 percent false alarm rate, and it flags crises it never saw in training in 53 to 93 percent of their windows. It cannot reliably say which crisis a period resembles (balanced accuracy 0.39 against a chance level of 0.25), and the trained Transformer did no better than an untrained one or seven summary statistics. The adaptive allocator did not beat mean-variance in three of four settings. Its one apparent win came from an invalid train and test split that produced a constant gold position, and every Sharpe ratio difference had a bootstrap interval that included zero. We report these results as they are, together with the diagnostics that exposed them. The project is at technology readiness level 3 and relates to Sustainable Development Goals 8 and 9.

CHAPTER 1
INTRODUCTION

1.1 Background

Prices in a financial market do not follow one fixed pattern. A calm year, a slow bear market and a sudden crash each have their own level of volatility, and assets that usually move independently tend to fall together when a crisis arrives [1]. Three episodes from the last twenty-five years show how different these events can be. In the dot-com bust (March 2000 to October 2002) the Nasdaq-100, which we follow through the QQQ fund, lost about 78 percent of its value over two and a half years. In the 2008 global financial crisis (October 2007 to March 2009) the S&P 500 fell by about 57 percent from peak to trough. The COVID-19 crash was far shorter: the S&P 500 dropped roughly 34 percent between 19 February and 23 March 2020 and then recovered faster than in any earlier bear market.

Most portfolio construction still starts from the mean-variance framework of Markowitz [2], which picks asset weights from the average returns and covariances observed in a past window. Those estimates are noisy, and when the market changes character the window is already out of date. DeMiguel and co-authors [3] found that splitting money equally across assets, the 1/N rule, is hard to beat out of sample, which tells us how little the estimated inputs can be trusted. Two lines of research try to fix this. Regime-switching models [4][5] let the data choose between a few market states, and reinforcement learning [6][7] lets an agent learn an allocation rule directly from prices.

This project sits where those ideas meet. We built a web application that takes live price data, labels the current market regime, compares any period the user selects with the three crises above, and tests whether an adaptive allocation rule beats simple benchmarks. During development we made the comparison with past crises the central feature, because it answers a question people ask in practice ("does this look like 2008?") and can be checked against evidence.

1.2 Motivations

Several things pushed us toward this topic.

- People reason about the present by analogy with past crises, but the analogy is usually informal. A tool can say how close a period is to each known crisis, on what evidence, and can say plainly when it resembles none of them.
- Static allocation rules assume stable market conditions. The reviewed papers (Table 2.1) report that adaptive methods do better, but few of them include transaction costs, repeat the training with several random seeds or guard against information leaking from the test period. We wanted to see how much of the claimed gain survives a stricter test.
- Free daily price data now goes back decades, which makes it possible to study all three crises with the same code and to run the application on live data.
- The project let us combine time series modelling, deep learning, clustering, numerical optimization and web development in one system, which is how such tools are built outside

the classroom.

1.3 Scope of the Project

The project covers daily data for liquid exchange-traded funds and a few other instruments. It delivers a back end, a dashboard and an evaluation of the models behind them. Table 1.1 lists what the work includes and what it leaves out.

Table 1.1 Scope of the project.

Within the scope	Outside the scope
Daily adjusted close prices from Yahoo Finance, fetched live and cached	Intraday, tick-level or order-book data
Three historical case studies: dot-com bust, 2008 financial crisis, COVID-19 crash	Other crises such as 1987, 1998 or 2022, and non-US markets
Unsupervised regime labelling with a three-state hidden Markov model	Using the regime label as a trading signal or forecast
Comparison of any window of at least 20 trading days with the case studies, using a Transformer encoder and HDBSCAN	Predicting future prices or returns
Long-only portfolios of three or four assets, rebalanced every 21 trading days with a 5 basis point cost	Short selling, leverage, derivatives, taxes and a liquidity or market-impact model
Walk-forward backtests, bootstrap intervals, seed and feature ablations	Live or paper trading with real money
A browser dashboard that runs on a single machine	Multi-user deployment, authentication and cloud hosting

The work is a proof of concept at technology readiness level 3, as set out in Section 2.3. It is an educational and research tool. Nothing in the application or in this report is investment advice.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

We began with four recent studies on adaptive portfolio optimization, listed in Table 2.1. All four report that an adaptive reinforcement learning agent does better than static allocation, but they differ in architecture, assets and how much they say about costs and interpretability.

Table 2.1 Literature review of recent adaptive portfolio optimization studies.

Paper	Objective	Methodology	Pros and cons	Findings
Zhang (2025), GAT multi-agent DRL [8]	Adapt to time-varying asset correlations	Graph attention network with three specialized RL agents	High returns; high complexity	Outperforms mean-variance and equal-weight
DRL-APO framework (2025) [9]	Adaptive optimization for dynamic markets	CNN-LSTM encoder with PPO policy gradient	Captures temporal patterns; price-only inputs	More adaptive than static models
RL-APM / DDPG (2025) [10]	Real-time single-stock allocation	DDPG agent with technical indicators (SMA, RSI, MACD)	Continuous control; ignores costs and liquidity	Improves on static baselines
Regime-aware RL (2025) [11]	Regime-adaptive, explainable allocation	Transformer-PPO and LSTM with regime detection	Best returns; LSTM more interpretable	Regime context improves stability

Beyond these four, we read the work our methods rest on. Markowitz [2] gave the mean-variance framework, and Sharpe [12] defined the return-to-risk ratio we use to score portfolios. DeMiguel et al. [3] compared fourteen optimized strategies with equal weighting and found that none consistently beat it, so we use both as baselines.

For regimes, Hamilton [4] introduced Markov-switching models for series whose behavior changes between states, and Ang and Bekaert [5] showed that such regimes matter for asset allocation. Hidden Markov models [13] are the standard way to fit these states without labels, and we use the hmmlearn implementation [14].

Reinforcement learning for trading goes back to Moody and Saffell [6]. Jiang et al. [7] applied deep networks to portfolio management, and DDPG [15] and PPO [16] are the usual training algorithms in the papers above. Salimans et al. [17] showed that evolution strategies, which estimate a gradient from the rewards of randomly perturbed parameters, can train policies without back-propagation. We chose evolution strategies for the allocator because they need no automatic differentiation and finish training in seconds on a laptop.

The Transformer [18] relies on self-attention instead of recurrence. Zerveas et al. [19] showed that Transformer encoders can learn representations of multivariate time series without labels, and we follow that idea at a small scale. HDBSCAN [20][21] clusters points of varying density and marks sparse points as noise, which suits the question of whether a window belongs to any known group at all. We use the scikit-learn implementation [22].

2.2 Research Gap

Comparing the four studies shows four gaps, summarized in Table 2.2 together with the focus we

gave each one and how far the finished project went.

Table 2.2 Research gaps, proposed focus and status.

Gap area	Limitation in existing work	Proposed focus of this work	Status in this project
Model adaptivity	Most RL models assume a single asset or a simplified setting and ignore transaction costs and liquidity constraints	Multi-asset framework with transaction costs and liquidity in the reward function	Multi-asset and transaction costs done; liquidity not modelled
Regime awareness	Static mean-variance and early RL models do not detect or respond to shifting regimes in real time	Explicit regime-detection module feeding the allocation decision	Done with an HMM; ablation shows no consistent gain from it
Risk-adjusted evaluation	Many studies report returns without risk-adjusted benchmarking across varied conditions	Backtests with Sharpe ratio, maximum drawdown and volatility-based metrics	Done, plus seeds, bootstrap intervals and ablations
Interpretability versus performance	High-performing deep RL models such as Transformer-PPO give up interpretability, which limits adoption	Balance predictive performance with explainable decision outputs	Regime view and comparison report are explainable; the allocation policy is not

A fifth gap emerged while reviewing the material. None of the studies offers a way to compare an arbitrary market period with named past crises and report how strong the resemblance is. We added that as a feature, and it became the part of the project we evaluated most carefully.

2.3 Objectives

The aim set out at the first review was to design and evaluate an adaptive portfolio optimization framework that reallocates assets in response to market regime shifts and improves risk-adjusted returns over traditional static strategies. The framework should reduce reliance on the fixed distributional assumptions of mean-variance models, support real-time data-driven rebalancing validated by backtesting on historical multi-asset data, and be benchmarked and interpretable enough for portfolio managers and researchers to use. The objectives were:

1. Develop an adaptive allocation model for multi-asset portfolios (proposed as a deep reinforcement learning model).
2. Integrate real-time market regime detection to trigger dynamic portfolio reallocation.
3. Incorporate transaction costs and liquidity constraints into the reward function for realistic decisions.
4. Benchmark against static strategies (mean-variance and equal-weight) using Sharpe ratio, drawdown and returns.
5. Validate robustness across bull, bear and volatile market regimes using historical financial data.

Two further objectives were added during development:

- Build a historical-pattern comparison that shows whether a chosen period follows the same trend as the dot-com bust, the 2008 crisis or the COVID-19 crash, using a Transformer encoder and HDBSCAN, and gives a normal-regime baseline when no crisis matches.

- Package everything as a working web application that uses live market data and explains its output.

Sustainable Development Goals. The project was mapped to SDG 8 and SDG 9 [23]. Table 2.3 states how, and the link is indirect: the work is a research prototype and does not itself change any economic outcome.

Table 2.3 Sustainable Development Goals addressed.

Goal	Connection to the project
SDG 8: Decent work and economic growth	Clear, evidence-based risk tools help investors and fund managers avoid large losses in crises, which supports stable economic activity. The tool is free and runs on a laptop.
SDG 9: Industry, innovation and infrastructure	The project applies machine learning to financial infrastructure and releases the code and evaluation scripts as open source for other researchers.

Expected outcomes. A conference paper or a journal article indexed in Scopus. A full manuscript with the complete evaluation has been prepared for that purpose.

Technology readiness level. The project is assigned TRL 3, analytical and experimental proof of concept [24]. The regime detection and adaptive rebalancing functions were validated through simulation on historical multi-asset data. They have not been tested in a live trading or production environment. The target for future work is TRL 4 to 5, validation in a relevant environment with live or paper-traded data.

2.4 Problem Statement

Traditional portfolio models such as mean-variance optimization and static allocation assume fixed market conditions. Real markets are non-stationary and move between regimes, so these models adapt slowly when volatility and correlations change. We therefore study two connected problems on daily multi-asset price data.

- **Historical-analogue comparison.** Given any symbol and date range of at least 20 trading days, decide whether the price path resembles the dot-com bust, the 2008 crisis, the COVID-19 crash or an ordinary period. Report the evidence: a similarity confidence, a chart overlay with a real historical window, the return, volatility and drawdown of the period, its regime composition, and a normal-regime baseline when nothing matches.
- **Regime-aware allocation.** Decide whether a policy that observes regime probabilities and asset momentum and volatility improves risk-adjusted performance over equal-weight and rolling mean-variance portfolios when it is trained only on data before a crisis and evaluated through the crisis and recovery, with transaction costs included.

2.5 Project Plan

The four-month plan presented at the first review (17 August 2026) has six tasks that overlap in time. Figure 2.1 shows the schedule and Table 2.4 gives the week of each task.

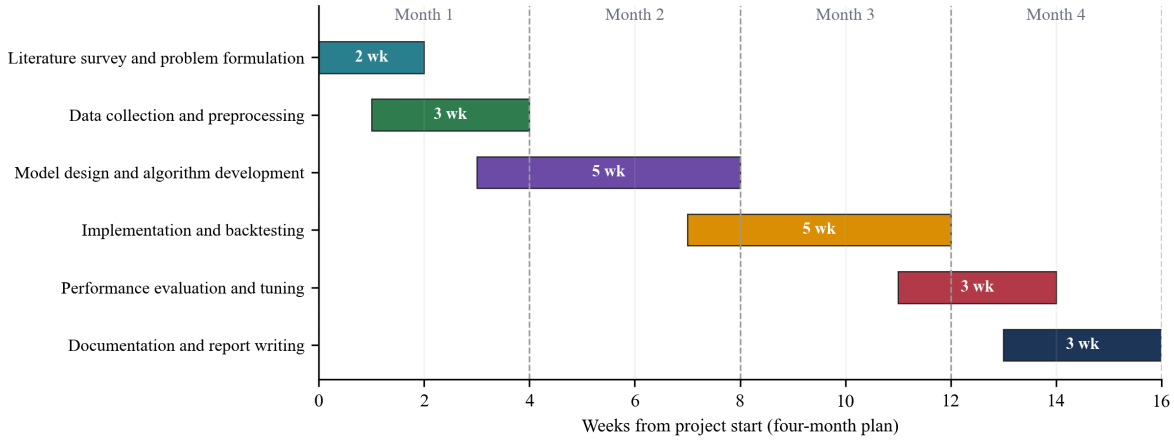


Figure 2.1 Four-month project work plan.

Table 2.4 Project plan with task durations.

No.	Task	Start (week)	Duration (weeks)	End (week)
1	Literature survey and problem formulation	0	2	2
2	Data collection and preprocessing	1	3	4
3	Model design and algorithm development	3	5	8
4	Implementation and backtesting	7	5	12
5	Performance evaluation and tuning	11	3	14
6	Documentation and report writing	13	3	16

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

Table 3.1 lists what the system must do and where each requirement is implemented.

Table 3.1 Functional requirements.

ID	Requirement	Implemented by
FR1	Show live quotes for SPY, QQQ, GLD, TLT, BTC-USD and the VIX index in a ticker ribbon	GET /api/market/live; TickerRibbon
FR2	Fetch daily history for any symbol and range, cache it, retry on failure, and fall back to labelled synthetic data for the three case studies if the feed is unreachable	market_data.py; synthetic_fallback.py
FR3	Classify each trading day of a symbol and range as bull, bear or crisis and plot the result over the price	GET /api/regimes/history; RegimeExplorer
FR4	Report the current regime of a symbol with its probability	GET /api/regimes/current
FR5	Compare a window of at least 20 trading days with the dot-com, 2008 and COVID-19 events and a normal baseline, returning confidence, overlay path, statistics and written insights	GET /api/regimes/compare; event_matcher.py
FR6	Offer an auto-detect mode and a mode that scores the window against one chosen event	event_id parameter; EventComparisonReport
FR7	Run a backtest of Equal-Weight, Mean-Variance and Adaptive strategies for a case study or for custom assets and dates	POST /api/backtest; engine.py
FR8	Let the user set rebalance frequency, cost in basis points, and the training generations and population of the adaptive policy	BacktestRequest schema
FR9	Display value curves, a metrics table (total and annualized return, volatility, Sharpe, maximum drawdown, Calmar) and allocation weights over time	BacktestPanel
FR10	Flag results that come from synthetic fallback data	synthetic_data field in the response
FR11	Validate inputs and return clear error messages for missing dates, short windows, unknown case studies and unreachable data	main.py exception handling
FR12	Describe the method and the technology stack inside the application	AboutPanel

3.1.2 Non-Functional

Table 3.2 Non-functional requirements.

Attribute	Requirement	Measured or observed
Performance	A comparison on a cached matcher should feel instant; a backtest should finish within a few seconds to tens of seconds	Comparison about 0.1 s after the one-time matcher build of about 25 s; backtest 8 to 17 s
Reliability	A rate limit or outage at the data provider must not stop the demonstration	Retry with back-off, 15-minute disk cache, labelled synthetic fallback
Reproducibility	Reported numbers must be repeatable	Fixed seeds; experiment scripts and saved outputs in paper/
Usability	One page with three tabs, no sign-up and no API key	Single-page React dashboard, dark theme
Interpretability	Output should show its evidence	Overlay with a real historical window, written insights, confidence with stated basis
Portability	Run on a common developer machine	macOS tested; Python 3.12 and Node 22; no GPU needed
Maintainability	Separate concerns so that a model can be replaced	Packages: data, ml, backtest; about 3,200 lines in total
Security and privacy	No personal data and no secrets	Only public market data is fetched; no accounts. Cross-origin access is open because the tool is meant for local use
Scalability	Single-user use	In-memory caches in one process; not built for many concurrent users

3.2 Feasibility Study

3.2.1 Technical Feasibility

Every component we needed exists as mature open-source software: scikit-learn and hmmlearn for the hidden Markov model and HDBSCAN, PyTorch for the Transformer encoder, SciPy for the mean-variance optimizer, FastAPI for the service layer and React for the interface. The Transformer is small (32 hidden units, two layers) and the reference corpus has under a thousand windows, so training takes seconds on a CPU. The whole pipeline, including a full set of experiments, ran on a single Apple M3 laptop with 8 GB of memory. The part that was less certain was data access: Yahoo Finance rate-limited our early requests, and we solved it with a direct client, caching and a labelled fallback (Section 5.2.5).

3.2.2 Economic Feasibility

The project needed no cash outlay. Table 3.3 lists the resources and what each cost.

Table 3.3 Resources and costs.

Resource	Source	Cost
Development machine (Apple M3, 8 GB)	Already owned by a team member	No additional cost
Python 3.12, FastAPI, PyTorch, scikit-learn, hmmlearn, SciPy, pandas, NumPy	Open-source licences	Free
Node.js, React, Vite, Recharts, axios	Open-source licences	Free
Daily market data	Yahoo Finance public chart endpoint, no API key	Free
Hosting	Local machine at 127.0.0.1	Free
GPU or cloud compute	Not used	None
Team effort	Three students over 16 weeks (Section 2.5)	Student time

A production service would add recurring costs for a licensed real-time data feed, hosting and monitoring. We did not estimate those because the project is not meant for production at this stage.

3.2.3 Social Feasibility

The tool helps students and analysts see how today's market compares with past crises, and it is open about its limits. That openness matters socially: a confident-looking score can lead a user to risk money they cannot afford to lose. The dashboard therefore labels its output as illustrative and not predictive, shows when data is synthetic, and states the confidence value with the evidence behind it. The application collects no personal data. The main risk is misuse of the output as investment advice, which the interface and this report both discourage.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.4 Hardware used for development and testing.

Component	Specification
Processor	Apple M3, 8 cores
Memory	8 GB unified memory
Storage	Under 1 GB for the code, dependencies and cached price files
GPU	Not required; the Transformer runs on the CPU
Network	Internet access for Yahoo Finance (the case studies also run offline on cached or synthetic data)
Client	Any current desktop browser

3.3.2 Software Specification

Table 3.5 Software stack.

Layer	Software	Version
Operating system	macOS	26.6.2
Language (back end)	Python	3.12.7
Web framework	FastAPI with Uvicorn; Pydantic	0.115.6; 0.34.0; 2.10.4
Numerical computing	NumPy, pandas, SciPy	1.26.4, 2.2.3, 1.13.1
Machine learning	scikit-learn (HDBSCAN, PCA, t-SNE, k-NN), hmmlearn, PyTorch	1.5.2, 0.3.3, 2.5.1
Front end	React, Vite, Recharts, axios	19.2, 8.2, 3.10, 1.20
Runtime (front end)	Node.js	22
Data source	Yahoo Finance chart endpoint, called through curl	n/a

The requirements file also lists the yfinance package, but the code does not use it. The data client talks to the Yahoo chart endpoint directly so that retries and caching are under our control.

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 System Architecture

The application has four layers, shown in Figure 4.1. The React dashboard runs in the browser and talks to a FastAPI service over eight REST endpoints. The service calls the analytics layer, which holds the hidden Markov model, the pattern matcher, the adaptive allocator and the backtest engine. The data layer fetches prices from Yahoo Finance, stores them in a CSV cache and, for the three case studies only, can fall back to a synthetic series calibrated to each crisis. The matcher is built once per server process, which takes about 25 seconds, and is reused afterwards. Backtest results are cached by a hash of their settings.

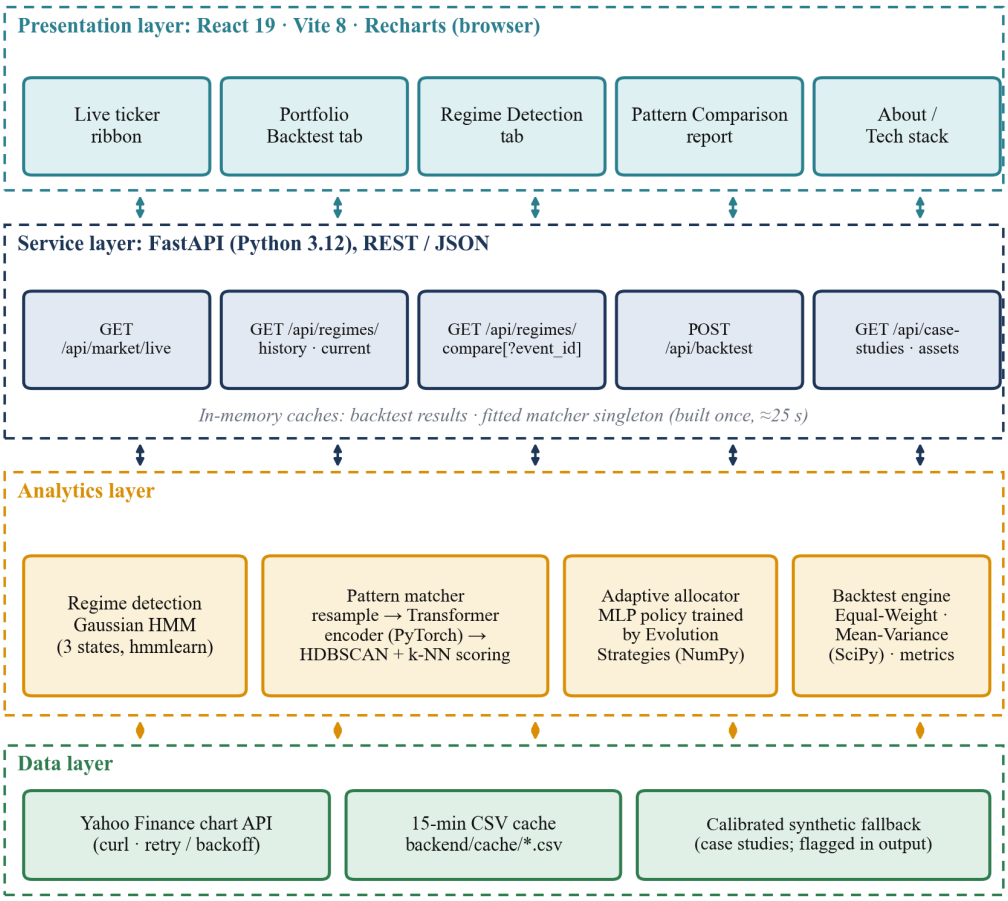


Figure 4.1 Layered architecture of the Adaptive Portfolio Optimizer.

Table 4.1 REST endpoints of the back end.

Method and path	Purpose	Main parameters
GET /api/health	Liveness check	none
GET /api/case-studies	Definitions of the three crises (dates, assets, description)	none
GET /api/assets/default	Default four-asset universe (SPY, QQQ, GLD, TLT)	none
GET /api/market/live	Latest quotes for the ticker ribbon	none
GET /api/regimes/current	Current regime and probabilities of a symbol	symbol, lookback_days
GET /api/regimes/history	Daily regime labels and probabilities	symbol, start, end
GET /api/regimes/compare	Historical-pattern comparison report	symbol, start, end, event_id (optional)
POST /api/backtest	Walk-forward backtest of three strategies	case_study or assets, start, end, rebalance_freq, cost_bps, es_generations, es_population

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 is the context diagram. The user sends a symbol, a date range and optional settings, and receives charts, a comparison report and backtest results. Yahoo Finance is the only external data source.

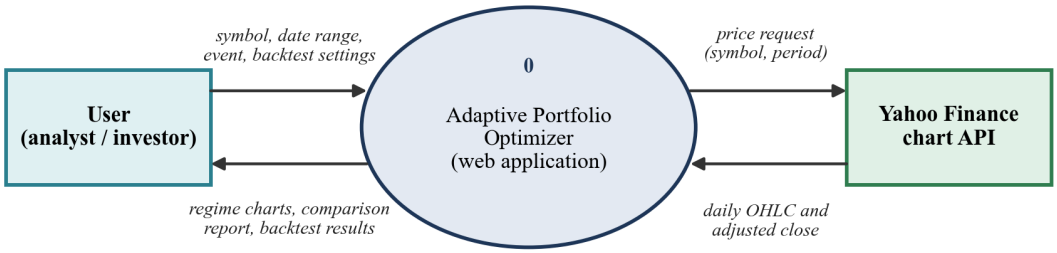


Figure 4.2 Data flow diagram, level 0 (context diagram).

Figure 4.3 breaks the system into five processes. Process 1.0 acquires and cleans prices and writes them to the price cache (D1). Process 2.0 fits the hidden Markov model and returns regime labels. Process 3.0 compares a window with the reference corpus and the fitted matcher stored in D2. Process 4.0 runs the backtest and keeps finished results in D3. Process 5.0 is the dashboard, which collects the user's input and presents all outputs.

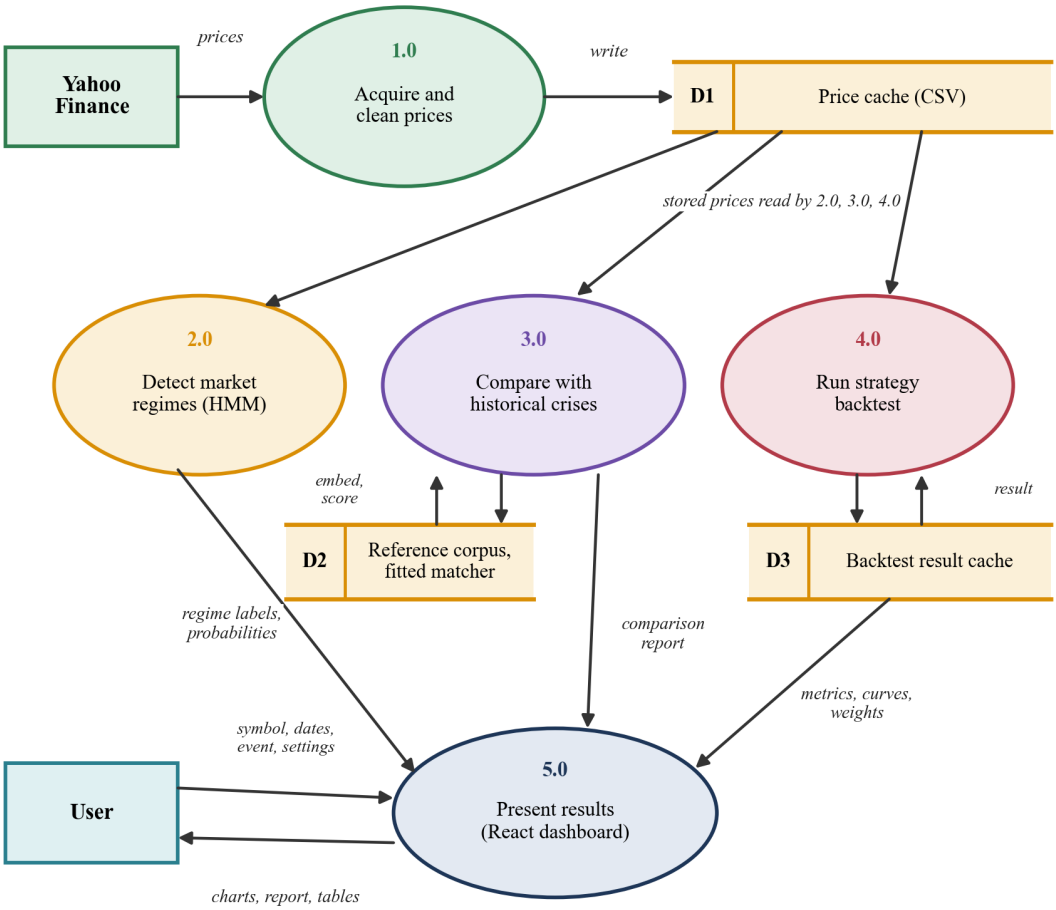


Figure 4.3 Data flow diagram, level 1.

4.2.2 Use Case Diagram

Figure 4.4 shows the two actors. The user can view the market ribbon and regime charts, compare a date range with historical crises (optionally targeting one event), read the report, run backtests and inspect their allocations. Yahoo Finance is a secondary actor that supplies prices to the use cases that need them. The most important use case is the comparison: the user picks a symbol and dates, optionally clicks one of the three event buttons, and receives a verdict with its evidence.

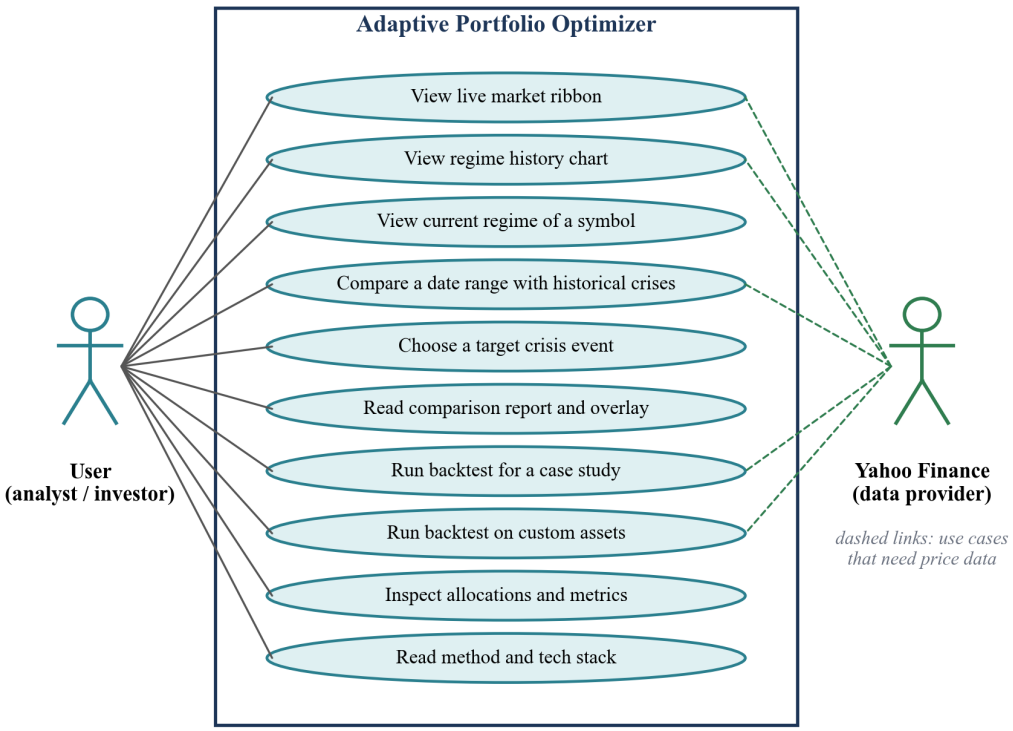


Figure 4.4 Use case diagram.

4.2.3 Class Diagram

Figure 4.5 shows the main data structures and modules of the back end. The API module receives a BacktestRequest and calls the market data, regime, matcher and backtest modules. A Matcher object bundles the encoder, the scaling constants, the list of CorpusWindow records, their embeddings and the cluster summaries. PolicyNet is the small neural policy used by the adaptive strategy, and ShapeAutoencoder wraps the ShapeEncoder with a decoder that is used only for training.

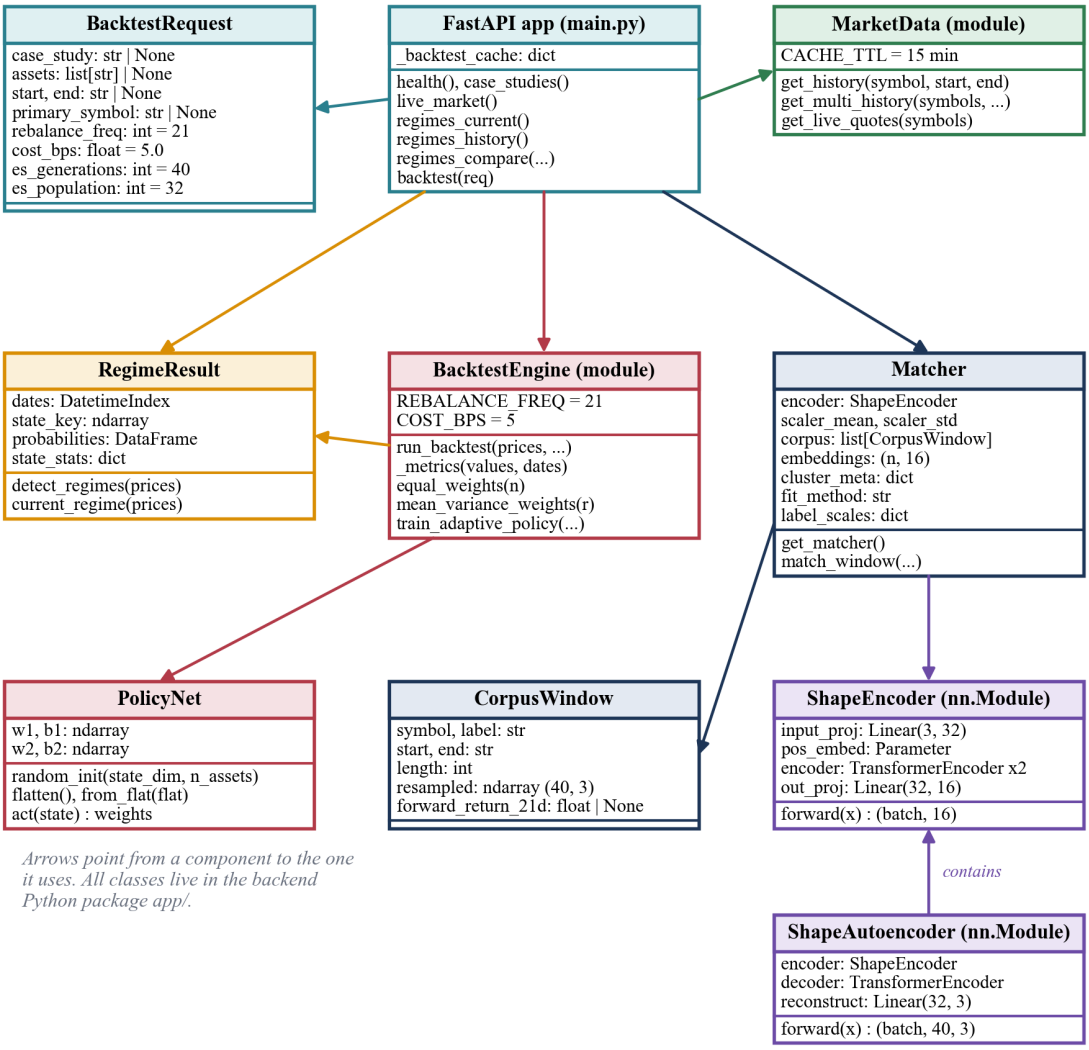


Figure 4.5 Class diagram of the back end.

4.2.4 Sequence Diagram

Figure 4.6 follows one comparison request. The dashboard calls the compare route, which hands the work to the matcher. The matcher gets prices through the data layer, converts the window into a 40 by 3 shape, asks the Transformer for an embedding, scores the labels and returns a report that the dashboard draws.

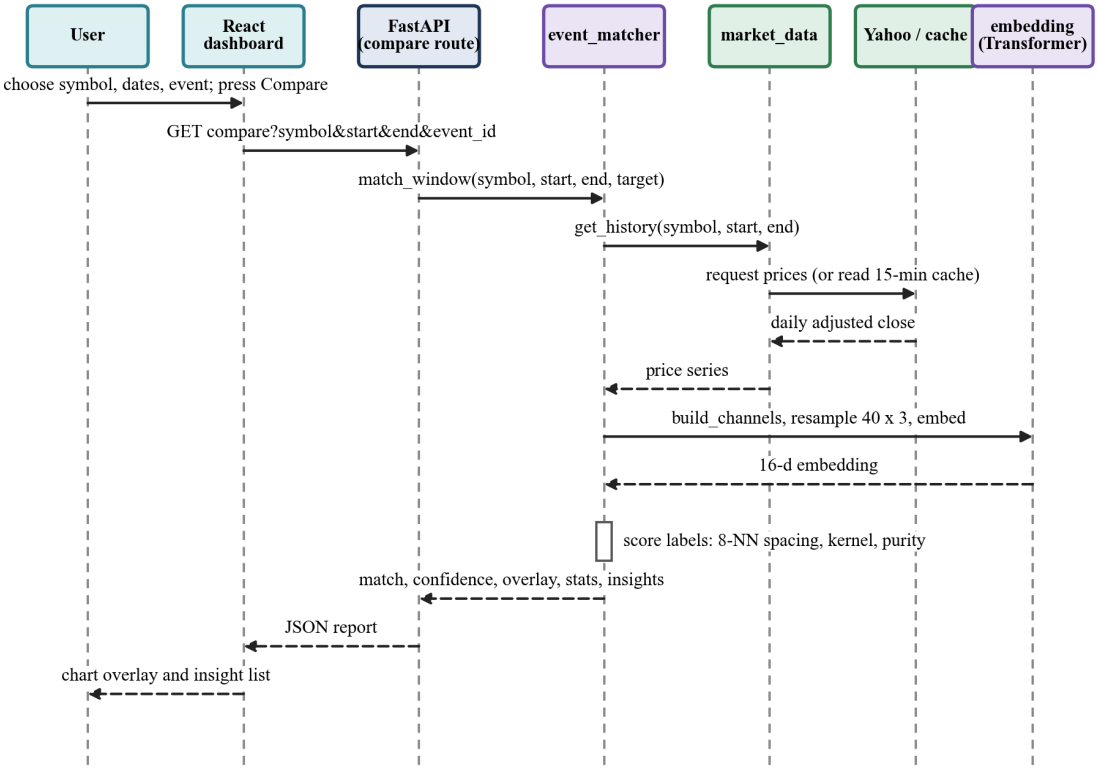


Figure 4.6 Sequence diagram of a historical-pattern comparison.

Figure 4.7 shows a backtest. After checking its cache, the service loads the price panel, fits the regime model on the primary symbol, trains the adaptive policy on the pre-crisis segment only, simulates the three strategies with the same rebalance dates and cost, and returns values, weights and metrics.

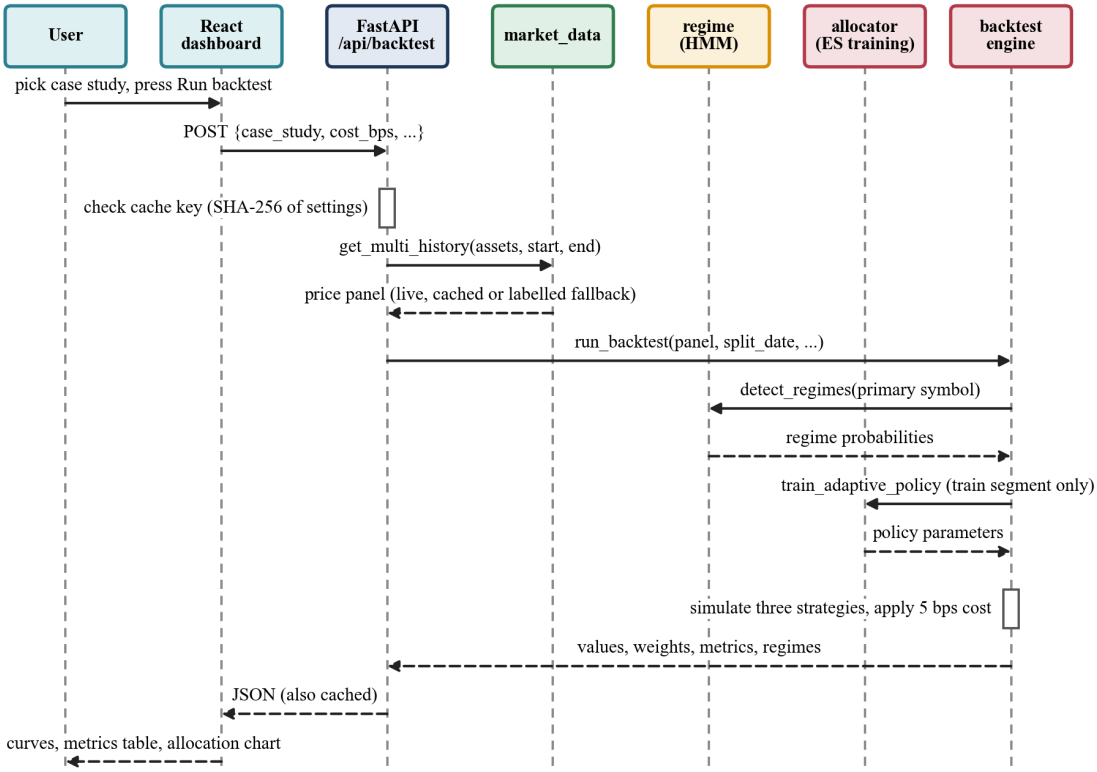


Figure 4.7 Sequence diagram of a portfolio backtest.

CHAPTER 5
METHODOLOGY AND TESTING

This chapter describes the modules of the system in the order data flows through them, and then the tests used to check the software and the models. Results are reported in Chapter 7.

5.1 Module Description

5.1.1 Data Acquisition and Preprocessing

All results use real daily adjusted close prices from the Yahoo Finance chart interface [25]. The client in market_data.py calls the endpoint through curl with explicit start and end timestamps. Yahoo's default "max" range quietly returns roughly monthly bars for long histories, so explicit timestamps are needed to get daily data. Failed requests are retried with back-off, and every response is cached on disk for 15 minutes. If the feed cannot be reached, the three case studies can run on a synthetic series calibrated to the size of each crisis, and the response carries a flag so the dashboard can say so. No result in this report used the synthetic fallback.

Table 5.1 defines the three crises and the four backtest settings. The dot-com setting appears twice. The version labelled "as deployed" uses gold futures (GC=F), whose Yahoo history only begins in August 2000, after the crisis had started. The "corrected" version replaces them with the Treasury fund VUSTX, which has a long history, so that a real pre-crisis training period exists (Section 7.3).

Table 5.1 Backtest settings and crisis definitions.

Setting	Assets	Data window	Trading days	Crisis window	Train / test days
Dot-com (as deployed)	SPY, QQQ, GC=F	2000-08-31 to 2003-12-31	829	2000-03-10 to 2002-10-09	30 / 799
Dot-com (corrected)	SPY, QQQ, VUSTX	1999-06-02 to 2003-12-31	1153	2000-03-10 to 2002-10-09	196 / 957
2008 GFC	SPY, QQQ, GLD, TLT	2006-01-04 to 2010-12-31	1258	2007-10-09 to 2009-03-09	443 / 815
COVID-19	SPY, QQQ, GLD, TLT	2019-01-03 to 2021-12-31	756	2020-02-19 to 2020-04-07	283 / 473

Table 5.2 Descriptive statistics of each asset over each backtest window (252 trading days per year).

Setting	Asset	Period	N	Ann. return	Ann. volatility	Max drawdown
Dot-com (as deployed)	SPY	2000-08-30 to 2003-12-31	830	-7.3%	22.3%	-47.3%
Dot-com (as deployed)	QQQ	2000-08-30 to 2003-12-31	830	-26.1%	44.5%	-80.5%
Dot-com (as deployed)	GC=F	2000-08-30 to 2003-12-31	830	13.5%	14.7%	-15.2%
Dot-com (corrected)	SPY	1999-06-01 to 2003-12-31	1154	-2.0%	21.9%	-47.5%
Dot-com (corrected)	QQQ	1999-06-01 to 2003-12-31	1154	-7.0%	44.8%	-83.0%
Dot-com (corrected)	VUSTX	1999-06-01 to 2003-12-31	1154	8.7%	9.5%	-12.8%
2008 GFC	SPY	2006-01-03 to 2010-12-31	1259	1.9%	24.9%	-55.2%
2008 GFC	QQQ	2006-01-03 to 2010-12-31	1259	6.2%	25.3%	-53.4%
2008 GFC	GLD	2006-01-03 to 2010-12-31	1259	21.2%	23.1%	-29.4%
2008 GFC	TLT	2006-01-03 to 2010-12-31	1259	4.9%	14.5%	-26.6%
COVID-19	SPY	2019-01-02 to 2021-12-31	757	25.9%	21.9%	-33.7%
COVID-19	QQQ	2019-01-02 to 2021-12-31	757	37.8%	24.9%	-28.6%
COVID-19	GLD	2019-01-02 to 2021-12-31	757	12.1%	15.3%	-18.8%
COVID-19	TLT	2019-01-02 to 2021-12-31	757	8.6%	16.3%	-21.3%

For the pattern comparison we built a reference corpus of 965 labelled windows. They come from the three case studies (QQQ for the dot-com bust, SPY for the other two) and from four calm periods: SPY in 2003 to 2006, 2012 to 2015 and 2016 to 2019, and QQQ in 2012 to 2015. The corpus holds 679 normal, 105 dot-com, 101 GFC and 80 COVID-19 windows.

Prices from different assets are aligned on shared trading dates, and daily simple returns are computed for the backtests. For regime detection, each day is described by three features: the log return, the 20-day annualized volatility and the drawdown from the 60-day rolling peak.

$$r_t = \ln \frac{P_t}{P_{t-1}} \tag{5.1}$$

$$\sigma_t = \sqrt{252}\,\mathrm{std}\,(r_{t-19}, \ldots, r_t) \tag{5.2}$$

$$d_t = \frac{P_t}{\max\limits_{s \in [t-59,\, t]} P_s} - 1 \tag{5.3}$$

The features are standardized before the model is fitted.

5.1.2 Regime Detection with a Hidden Markov Model

A three-state Gaussian hidden Markov model with diagonal covariance [13] is fitted by expectation-maximization (300 iterations, seed 7) on the standardized features. Each state has its own mean and variance for the three features, and a transition matrix describes how likely the market is to move between states.

$$z_t \mid z_{t-1} = i \sim \mathrm{Cat}(A_{i\cdot}), \qquad \mathbf{x}_t \mid z_t = j \sim \mathcal{N}\left(\mu_j, \mathrm{diag}(\sigma_j^2)\right) \tag{5.4}$$

State numbers are arbitrary, so we name the states afterwards from their statistics. The state with the highest average volatility is Crisis. Of the other two, the one with the higher average return is Bull and the other is Bear. The model never sees crash dates. Its posterior probabilities are used by the dashboard and as part of the state of the adaptive allocator.

5.1.3 Length-Invariant Shape and Transformer Encoder

The comparison has to treat a 33-day crash and a 600-day bear market on equal terms. For a window of n trading days we compute three channels from the window's own start: cumulative return, a 5-day rolling annualized volatility, and the running drawdown. Each channel is linearly interpolated onto 40 equally spaced points of a normalized time axis, so every window becomes a 40 by 3 array.

$$c_\tau = \frac{P_\tau}{P_1} - 1 \tag{5.5}$$

$$v_\tau = \sqrt{252}\,\mathrm{std}(\rho_{\tau-4:\tau}) \tag{5.6}$$

$$g_\tau = \frac{P_\tau}{\max_{s \le \tau} P_s} - 1, \qquad \tilde{X} \in \mathbb{R}^{40\times 3} \tag{5.7}$$

Here ρ stands for the daily returns inside the window. The channels are standardized with means and standard deviations computed over the corpus.

A small Transformer encoder [18] maps the 40 by 3 array to a 16-dimensional embedding (Figure 5.1). A linear layer lifts each time step to 32 dimensions and a learned positional embedding is added. Two encoder layers with four attention heads and a 64-unit feed-forward block (GELU activation [26], layer normalization [27]) follow. The 40 outputs are averaged and a last linear layer produces the 16 values. Self-attention is the scaled dot product of Vaswani et al.:

$$\mathrm{Attn}(Q,K,V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{5.8}$$

The encoder is trained without labels as an autoencoder. A decoder of the same size rebuilds the 40 by 3 array from the embedding, and training minimizes the squared reconstruction error with Adam [28] (learning rate 0.002, 120 full-batch epochs) in PyTorch [29]. The decoder is thrown

away after training.

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \left\| \tilde{X}_i - g_\phi\left(f_\theta(\tilde{X}_i)\right) \right\|_F^2 \tag{5.9}$$

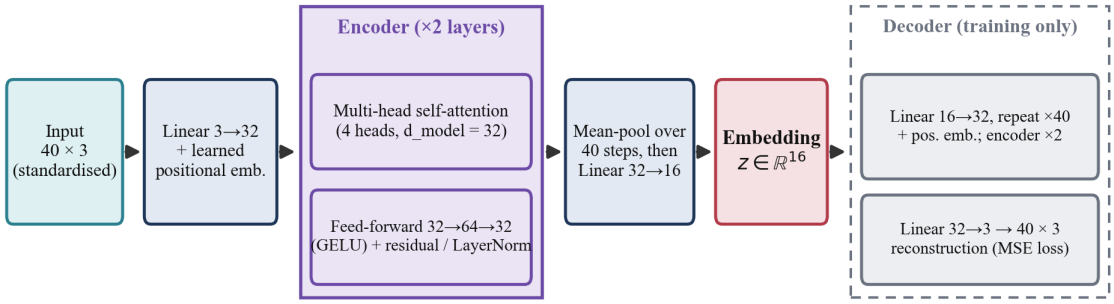


Figure 5.1 Transformer shape encoder and the decoder used only for training.

5.1.4 Reference Corpus and HDBSCAN

Overlapping windows are cut from each source series at 21 lengths between 10 and 600 trading days, with a stride of max(3, length/5) days. A window gets an event label if the crisis covers at least half of the window, or if the window contains at least 80 percent of the crisis and is no longer than four times its length. The second rule exists because the COVID-19 crash lasted only 33 trading days, so most windows that contain it are longer than the crash itself. Windows that overlap a crisis by 5 to 50 percent are dropped as ambiguous, and the rest are labelled normal. Each event class is capped at 120 windows and the normal class at 2.5 times the event total. The cap is stratified by window length, so a large number of short windows does not push out the rarer long ones.

HDBSCAN [20][21] is run on the embeddings over a grid of nine settings of minimum cluster size and minimum samples. We choose the setting that gives each event its own cluster with a majority of at least 60 percent, with high purity and little noise, and that avoids splitting into many small clusters. On the deployed corpus it finds 3 clusters. We use the clustering to inspect the structure of the corpus. The final score does not rely on cluster centers, because the center of an irregular density cluster can lie in empty space, and about 16 percent of the corpus is classed as noise that no center describes.

5.1.5 Similarity Scoring and the Comparison Report

For a query embedding z and a label ℓ, let $N_k(z)$ be the k = 8 corpus windows nearest to z that carry label ℓ, and let s_ℓ be the typical in-class spacing, the 75th percentile over the label's windows of the mean distance to their eight nearest same-label neighbors. The distance score m_ℓ is turned into a similarity by a Gaussian kernel, and a purity term p_ℓ measures how many of the 15 nearest windows overall carry the same label, weighted by inverse distance.

$$m_\ell(z) = \frac{1}{k} \sum_{j \in \mathcal{N}_k^\ell(z)} \|z - z_j\|_2 \tag{5.10}$$

$$\pi_\ell(z) = \exp\left[-\frac{1}{2}\left(\frac{m_\ell(z)}{s_\ell}\right)^2\right] \tag{5.11}$$

$$p_\ell(z) = \frac{\sum_{j \in \mathcal{N}_{15}(z)} w_j \mathbb{1}[y_j = \ell]}{\sum_{j \in \mathcal{N}_{15}(z)} w_j} \tag{5.12}$$

$$w_j = \frac{1}{\|z - z_j\|_2 + 10^{-3}} \tag{5.13}$$

$$C_\ell(z) = \pi_\ell(z)\, p_\ell(z) \tag{5.14}$$

In targeted mode the user picks an event and the confidence C_ℓ is reported for that event. In auto mode the label with the highest C_ℓ wins. A match requires $m_\ell \le 1.4\ s_\ell$ and $C_\ell \ge 0.20$, and in auto mode the winner must not be the normal class and must have p_ℓ above 0.6. If these conditions fail, the report says that no crisis resembles the period and gives the normal-regime confidence instead. Every report also contains the nearest same-label real window for the overlay, the annualized return, volatility and maximum drawdown of the period, the share of bull, bear and crisis days from the HMM, and the average 21-day forward return of the nearest same-label windows as a historical baseline. Figure 5.2 gives the whole pipeline.

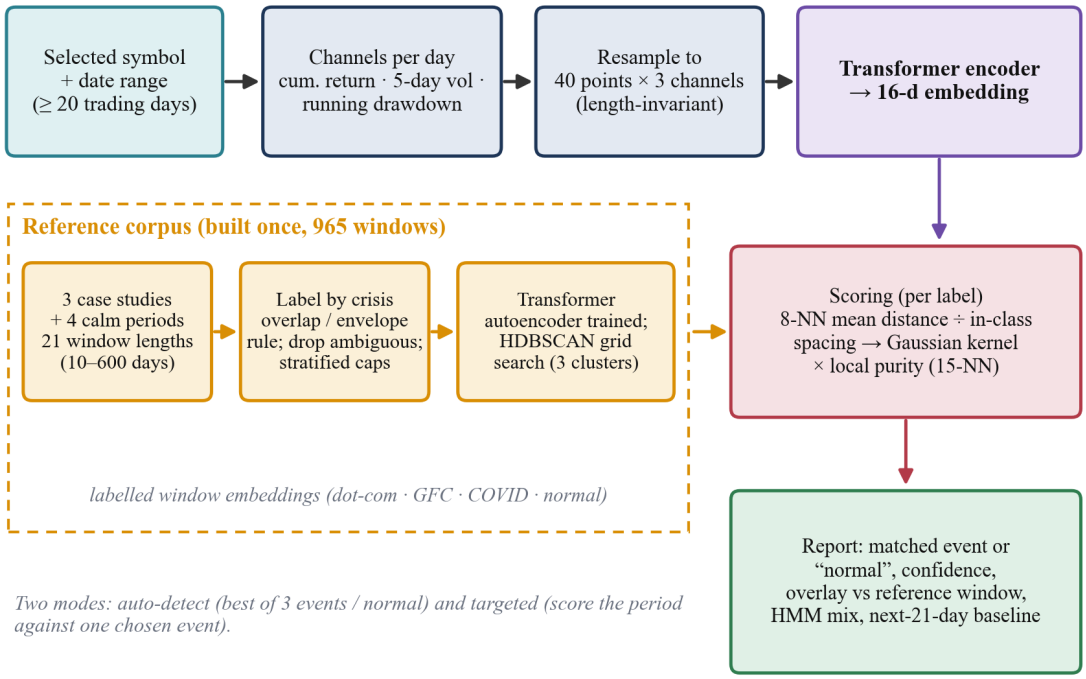


Figure 5.2 Historical-pattern comparison pipeline.

5.1.6 Allocation Strategies

Three strategies are compared on the same rebalance dates (every 21 trading days) and with the same proportional cost of 5 basis points on turnover.

- **Equal-Weight** holds 1/N of each asset.

- **Mean-Variance** maximizes the Sharpe ratio of a long-only, fully invested portfolio using the previous 63 trading days, solved with sequential least squares programming. If the solver fails it falls back to equal weights.
- **Adaptive** is a small neural policy. Its input is the state s_k: the three regime probabilities, and the 21-day momentum and 21-day annualized volatility of every asset. A hidden layer of 16 tanh units and a softmax output give long-only weights.

$$\mathbf{w}_k = \mathrm{softmax}(W_2 \tanh(W_1 \mathbf{s}_k + \mathbf{b}_1) + \mathbf{b}_2) \tag{5.15}$$

The policy is trained with evolution strategies [17]. In each of 40 generations, 32 antithetic Gaussian perturbations of the parameter vector ($\sigma = 0.1$) are scored by simulating the policy through the training segment. The rewards are rank-normalized and the parameters move along the reward-weighted perturbation direction with a step of 0.05. The reward subtracts a drawdown penalty and a turnover penalty from the Sharpe ratio, which is how transaction costs enter the objective.

$$R(\theta) = \mathrm{SR} - 2\,\mathrm{MDD} - 0.5\,\overline{\mathrm{TO}} \tag{5.16}$$

$$\theta \leftarrow \theta + \frac{\alpha}{N\sigma} \sum_{i=1}^{N} \hat{F}_i \varepsilon_i \tag{5.17}$$

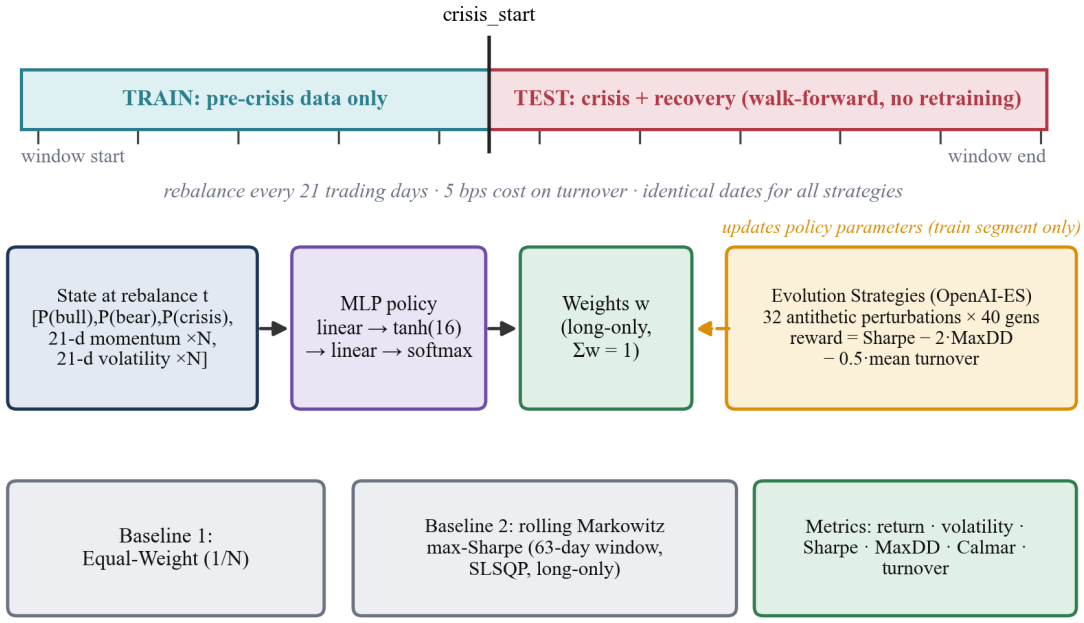


Figure 5.3 Walk-forward protocol: training on the pre-crisis segment, evaluation through the crisis.

5.1.7 Backtest Engine and Evaluation Metrics

The engine splits each setting at the crisis start. The adaptive policy and the 63-day look-back of the mean-variance strategy only ever use data before the date being decided. The test segment runs from the crisis start to the end of the data window. With daily portfolio returns r and portfolio value V, the metrics are:

$$\mathrm{SR} = \sqrt{252}\,\frac{\overline{r - r_f/252}}{\mathrm{std}(r)} \tag{5.18}$$

$$\mathrm{MDD} = \min_{t}\left(\frac{V_t}{\max\limits_{s \le t} V_s} - 1\right) \tag{5.19}$$

$$\mathrm{Calmar} = \frac{\mathrm{AnnRet}}{|\mathrm{MDD}|} \tag{5.20}$$

$$\mathrm{TO}_k = \|\mathbf{w}_k - \mathbf{w}_{k-1}\|_1 \tag{5.21}$$

$$V \leftarrow V(1 - c\,\mathrm{TO}_k), \qquad c = 5 \times 10^{-4} \tag{5.22}$$

$$\Delta = \mathrm{SR}_{\mathrm{adaptive}} - \mathrm{SR}_{\mathrm{baseline}} \tag{5.23}$$

The risk-free rate is 2 percent a year. Total return, annualized return, annualized volatility and mean turnover are also reported. For the matcher, the event classes are much smaller than the normal class, so we report balanced accuracy (the mean recall over the four classes, with a chance level of 0.25) and macro-F1, together with the crisis recall (the share of event windows predicted as some event) and the false-alarm rate (the share of normal windows predicted as an event). Plain accuracy is given only for completeness, since always answering "normal" already scores 70 percent.

5.1.8 Dashboard

The React interface has three tabs. Portfolio Backtest lets the user choose a case study, runs the backtest and draws the value curves, the metrics table and the allocation chart. Regime Detection draws the price with colored regime bands for any symbol and range, and below it the Historical Pattern Comparison report with its overlay chart, key figures and written insights. About describes the method and the stack. A ticker ribbon at the top polls the live quotes. The charts use Recharts, and a request counter in the regime explorer ignores answers to outdated requests. Table 5.3 collects the hyper-parameters of all modules.

Table 5.3 Hyper-parameters used in the experiments.

Component	Setting
HMM	3 states, diagonal covariance, 300 EM iterations, seed 7; features: log return, 20-day volatility, 60-day drawdown
Shape	40 points by 3 channels; windows of 10 to 600 days (21 lengths); stride max(3, length/5)
Transformer	d_{model} 32, 4 heads, 2 layers, feed-forward 64 (GELU), learned positions, mean pooling, 16-d output; Adam, learning rate 0.002, 120 epochs
Corpus	Event cap 120 per class (length-stratified); normal cap 2.5 times the events; 50 percent overlap or 80 percent coverage rule
HDBSCAN	9-point grid on minimum cluster size and minimum samples; the score favors one majority cluster per event, high purity, low noise
Scoring	k = 8 same-label neighbors; Gaussian kernel on spacing s; purity over 15 nearest; match if $m \leq 1.4$ s and $C \geq 0.20$
Backtest	Rebalance every 21 trading days; cost 5 bps on turnover; risk-free rate 2 percent; 252 trading days a year
Mean-Variance	63-day window, long-only maximum Sharpe (SLSQP), equal-weight fallback
ES policy	16 tanh hidden units; 32 antithetic perturbations by 40 generations; σ 0.1; step 0.05; reward SR - 2 MDD - 0.5 turnover

5.2 Testing

5.2.1 Approach and Test Environment

The project has no earlier automated test suite, so for this report we wrote a test script (paper/scripts/run_tests.py) with 21 test cases and ran it on the development machine described in Section 3.3. The API cases call the real routes through FastAPI's test client with live Yahoo data, so they also check the data path. The unit cases check numerical properties of single modules. Every case was run once and the observed result was recorded; all 21 passed.

5.2.2 Functional (API) Tests

Table 5.4 Functional tests of the REST API.

ID	Test case	Expected result	Observed result	Result
TC01	GET /api/health	HTTP 200	HTTP 200; body holds a status field	Pass
TC02	GET /api/case-studies	three case studies	HTTP 200; ids dotcom, gfc and covid	Pass
TC03	GET /api/assets/default	default four-asset universe	HTTP 200; assets SPY, QQQ, GLD, TLT	Pass
TC04	GET /api/market/live	at least one live quote	HTTP 200, 6 quotes	Pass
TC05	GET /api/regimes/ current (SPY)	regime label, probabilities sum to 1	HTTP 200; regime bear, probabilities sum to 1.000	Pass
TC06	GET /api/regimes/ history (SPY 2007-2010)	aligned arrays, states in {bull, bear, crisis}	HTTP 200; 988 daily labels, states bear, bull and crisis present	Pass
TC07	Compare without start and end dates	HTTP 400	HTTP 400, start and end are required	Pass
TC08	Compare window of 6 trading days	HTTP 400 (window too short)	HTTP 400; message says the window is too short and gives the count (7 trading days)	Pass
TC09	Compare COVID-19 window, targeted at covid	matched, event covid	HTTP 200; matched, event covid, confidence 0.59	Pass
TC10	Compare calm 2013-2015 window, auto mode	no crisis match	HTTP 200; no crisis matched, normal regime	Pass
TC11	Compare with an invalid ticker	HTTP 400 or 502	HTTP 502; message says Yahoo returned no data for that symbol	Pass
TC12	POST /api/backtest with unknown case study	HTTP 400	HTTP 400; message names the unknown case study	Pass
TC13	POST /api/backtest, COVID-19 case study	HTTP 200 with three strategies	HTTP 200; response holds strategies, metrics for the test and full periods, regimes and the split date	Pass

5.2.3 Unit Tests

Table 5.5 Unit tests of the analytics modules.

ID	Test case	Expected result	Observed result	Result
TC14	Backtest weights are fully invested	each rebalance row sums to 1	108 rebalance rows checked; largest deviation of a weight sum from 1 is 1.0e-04 (rounding in the output)	Pass
TC15	Higher cost lowers net return	return at 0 bps above return at 50 bps	Mean-Variance final value 1.705 at 0 bps vs 1.367 at 50 bps (start 1.0)	Pass
TC16	Resampling is length invariant	(40, 3) for any window length	(40, 3) for windows of 33, 120 and 600 days	Pass
TC17	Equal-weight and mean-variance weights	sum to 1, long-only	equal-weight sum 1.0000; mean-variance sum 1.0000 with minimum weight 0.0000	Pass
TC18	Adaptive policy output and parameter round trip	softmax weights, lossless flatten	softmax weights sum to 1.0000 with minimum 0.016; flatten and rebuild give identical output	Pass
TC19	HMM crisis state has highest volatility	crisis = max volatility state	mean volatility: crisis 0.543, bear 0.220, bull 0.131	Pass
TC20	Reference corpus and embedding size	965 windows, 16-d embeddings	embeddings of shape (965, 16); 679 normal, 105 dot-com, 101 GFC and 80 COVID-19 windows; 3 HDBSCAN clusters	Pass
TC21	Matcher is deterministic	identical confidence on repeat	two identical queries gave confidence 0.5912 both times	Pass

TC15 compares the final value of the Mean-Variance strategy in the 2008 setting at 0 and at 50 basis points of cost. TC19 uses the SPY history from 2007 to 2010 and checks that the state named Crisis really is the highest-volatility state, which holds by construction but guards against a regression in the relabelling code.

5.2.4 Model Validation Protocol

The software tests say nothing about whether the models are any good, so we designed four further experiments. Their results are in Chapter 7.

- **Blocked, purged cross-validation of the matcher.** Within each label the windows are sorted by start date and cut into five contiguous blocks. Any training window that overlaps in calendar time with a test window is removed, and the encoder is retrained on the training windows only. Ordinary shuffled cross-validation would be misleading here, because neighboring windows share most of their days.
- **Leave-one-event-out.** Train on two crises plus normal windows, purge anything overlapping the held-out crisis, and check whether the unseen crisis is flagged as any crisis

class rather than normal.

- **Embedding ablation.** The same 8-nearest-neighbor classifier is run on the trained Transformer, an untrained random Transformer, the raw 120-value shape, a 16-component PCA of it, and seven hand-made summary statistics. If the trained Transformer does not beat the others, it adds nothing.
- **Allocator ablation and bootstrap.** Six seeds, four settings and three variants (full, no regime information, untrained policy), plus a paired stationary bootstrap [30] (5,000 resamples, mean block length 10 days) of the Sharpe ratio difference between the adaptive policy and each baseline.

5.2.5 Defects Found and Fixed

Table 5.6 Defects found during development and testing.

Issue	Symptom	Cause	Fix
Data rate limiting	Early requests to Yahoo returned HTTP 429 in bursts	Default client and repeated requests for the same history	Direct curl client with a browser-like user agent, retries with back-off, 15-minute disk cache, labelled synthetic fallback
Coarse history	Long histories came back as roughly monthly bars	Yahoo's "max" range coarsens the interval	Explicit start and end timestamps in every request
Stale chart	The regime chart sometimes showed the previous selection	Older responses arrived after newer ones and overwrote them	Request counters in the regime explorer ignore outdated answers; load functions take explicit arguments
Shape mismatch	Dot-com queries were compared with a corpus built from a different symbol	Corpus used SPY while the shape of the event is clearest in QQQ	A shape_symbol field in the case-study configuration, used by both
Invalid dot-com split	The adaptive policy held 100 percent gold in every period	GC=F history starts in August 2000, leaving 30 training days	Documented as a failed setting; corrected setting uses VUSTX with 196 training days
Run-to-run drift	COVID-19 adaptive Sharpe moved from 0.85 to 1.05 with the same code and seed	Small revisions in refreshed Yahoo data change the HMM fit	Not removable; reported with seed ranges (Section 7.5)

CHAPTER 6
PROJECT DEMONSTRATION

6.1 Running the Application

The project needs Python 3.12 and Node.js, and it needs no API key. From the project folder one script installs the dependencies on first use and starts both servers.

```
./run.sh
# back end:  http://127.0.0.1:8000   (interactive API docs at /docs)
# dashboard: http://127.0.0.1:5173
```

The first comparison after the server starts takes about 25 seconds, because the matcher trains the encoder and builds the corpus. The page says so while it waits. Later comparisons take about a tenth of a second.

6.2 Portfolio Backtest Tab

The user picks one of the three case studies and presses Run Backtest. Figure 6.1 shows the 2008 crisis. The upper chart gives the value of one unit invested in each strategy. The dashed line marks the end of the adaptive policy's training data and the red band is the crisis window. Below the chart, the metrics table can switch between the out-of-sample segment and the full period, and the last panel stacks the weights the adaptive policy chose at each rebalance. In this run the policy moved almost entirely into long Treasuries (TLT) in late 2008.

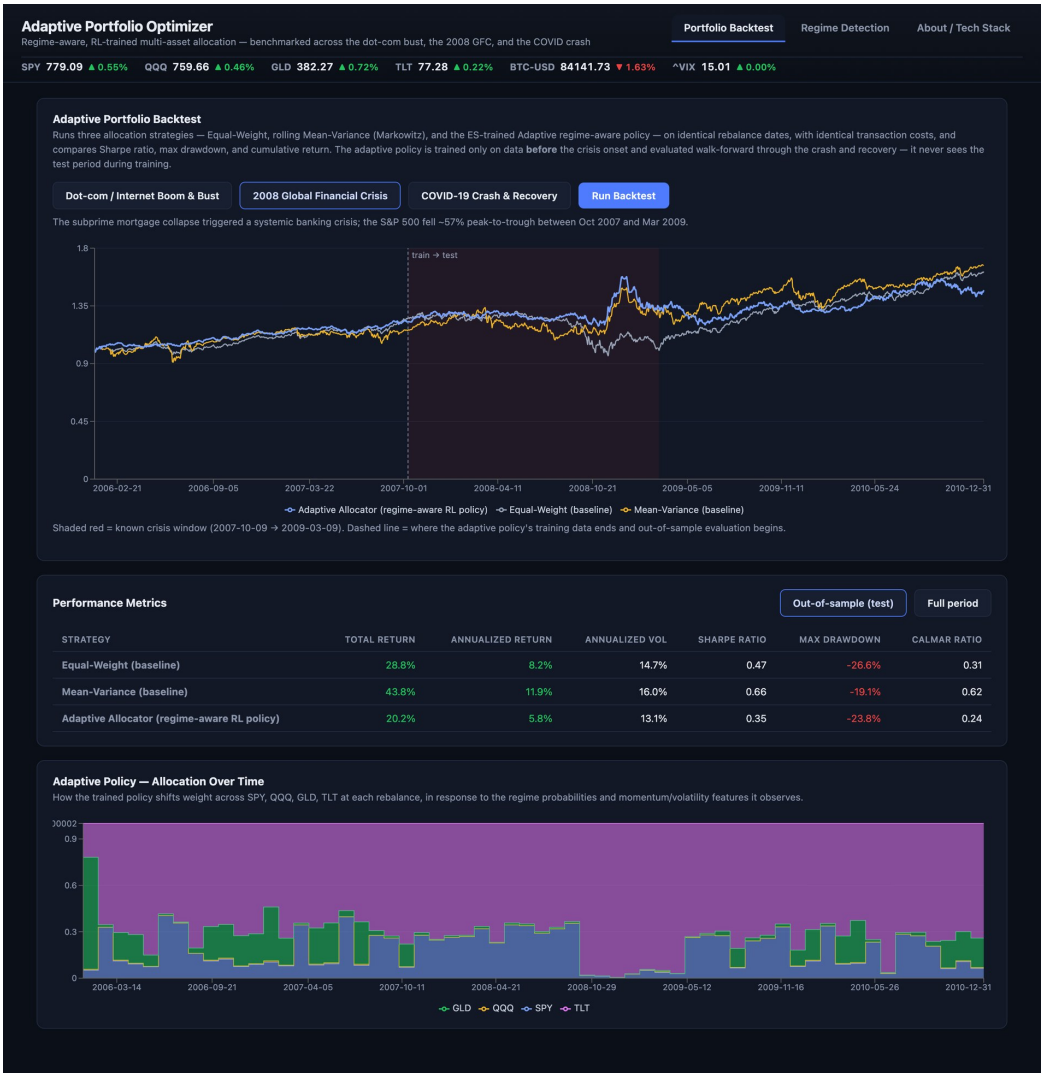


Figure 6.1 Portfolio Backtest tab for the 2008 crisis.

6.3 Regime Detection Tab

The upper panel colors each trading day of the chosen symbol and range by regime. Figure 6.2 shows SPY from 2019 to 2021. The red band is the COVID-19 crash and amber and green bands alternate before and after it. The panel beneath is the Historical Pattern Comparison. In this capture it is still computing, which is what the user sees on the first comparison after startup.

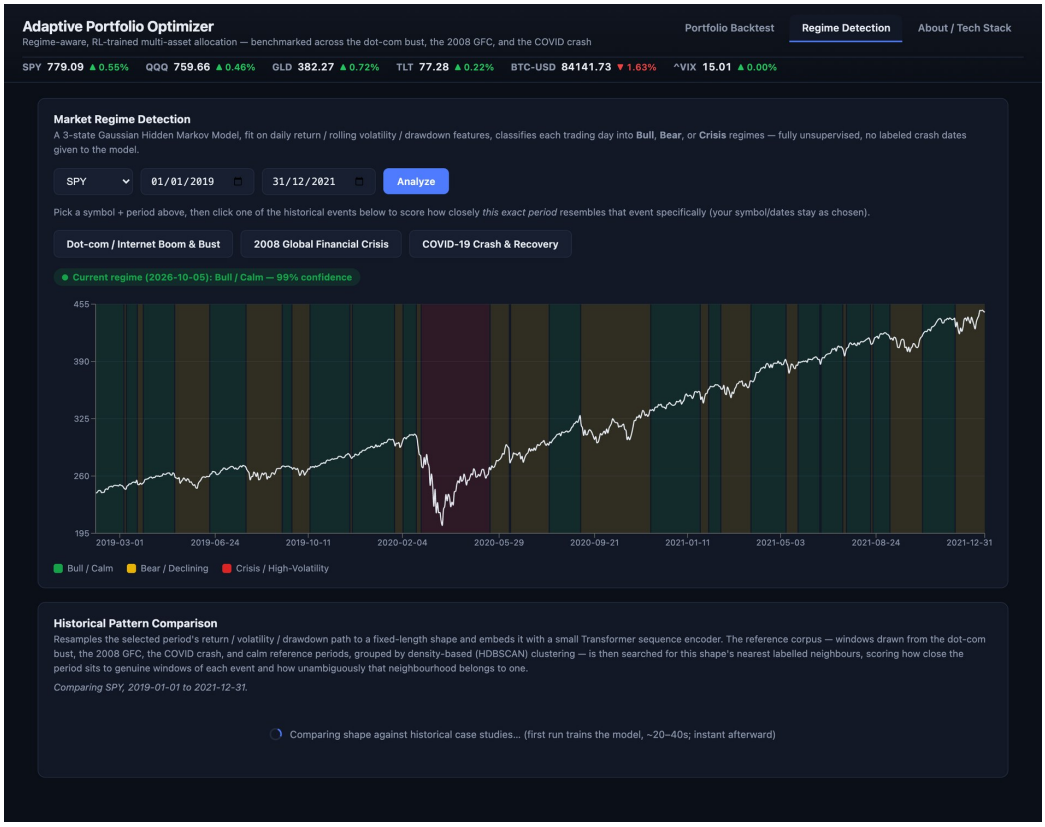


Figure 6.2 Regime Detection tab for SPY, 2019 to 2021, with the comparison loading.

When the user selects the dates 15 January to 15 May 2020 and clicks the COVID-19 button, the report in Figure 6.3 appears. The 85-trading-day window is judged to closely resemble the COVID-19 crash with 59 percent confidence. The overlay chart draws the selected period (blue) against a real COVID-19 reference window (purple, dashed). Four figures sit under it: annualized return of -32.1 percent, annualized volatility of 51.9 percent, a maximum drawdown of -33.7 percent and the match confidence. The written insights state that the HMM labelled 32 percent of the days Bull, 38 percent Bear and 29 percent Crisis, and that similar historical windows were followed by an average 21-day return of +5.2 percent (standard deviation 3.2 percent, 8 windows). The footer reminds the reader that the analysis is illustrative and not investment advice.

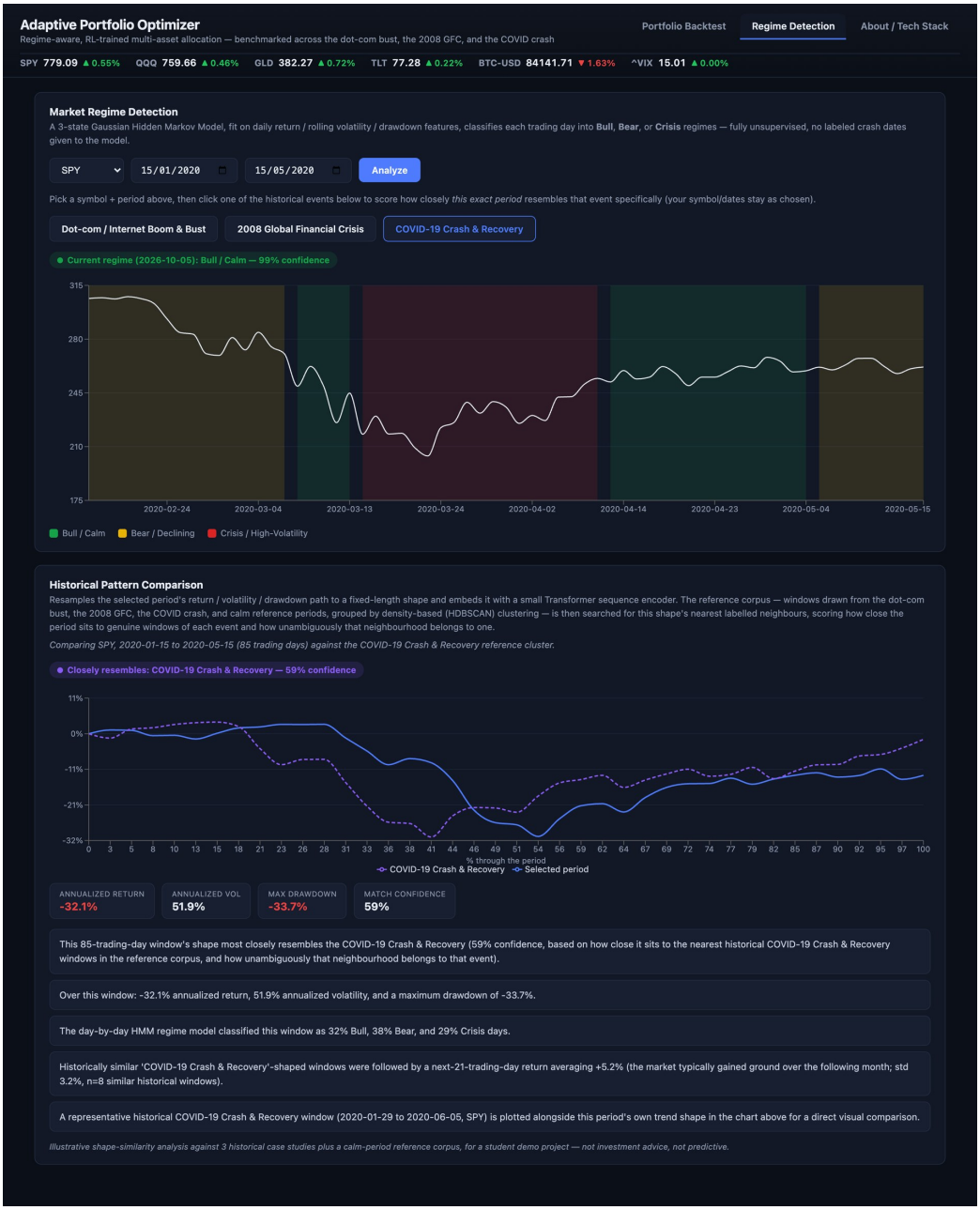


Figure 6.3 Regime Detection tab with the comparison report for the COVID-19 window.

If a period matches no event, the same panel states that the period tracks none of the three crises, shows the confidence for the normal class and gives the normal-regime baseline in place of an event overlay. The 2013 to 2015 SPY window is an example (Section 7.2).

6.4 About Tab

The About tab in Figure 6.4 explains what the application does, lists the stack, walks through the pipeline and states the scope limits. One thing in it is out of date: its description of the match step still mentions nearest cluster centers, whereas the final scoring uses the nearest labelled windows (Section 5.1.5).

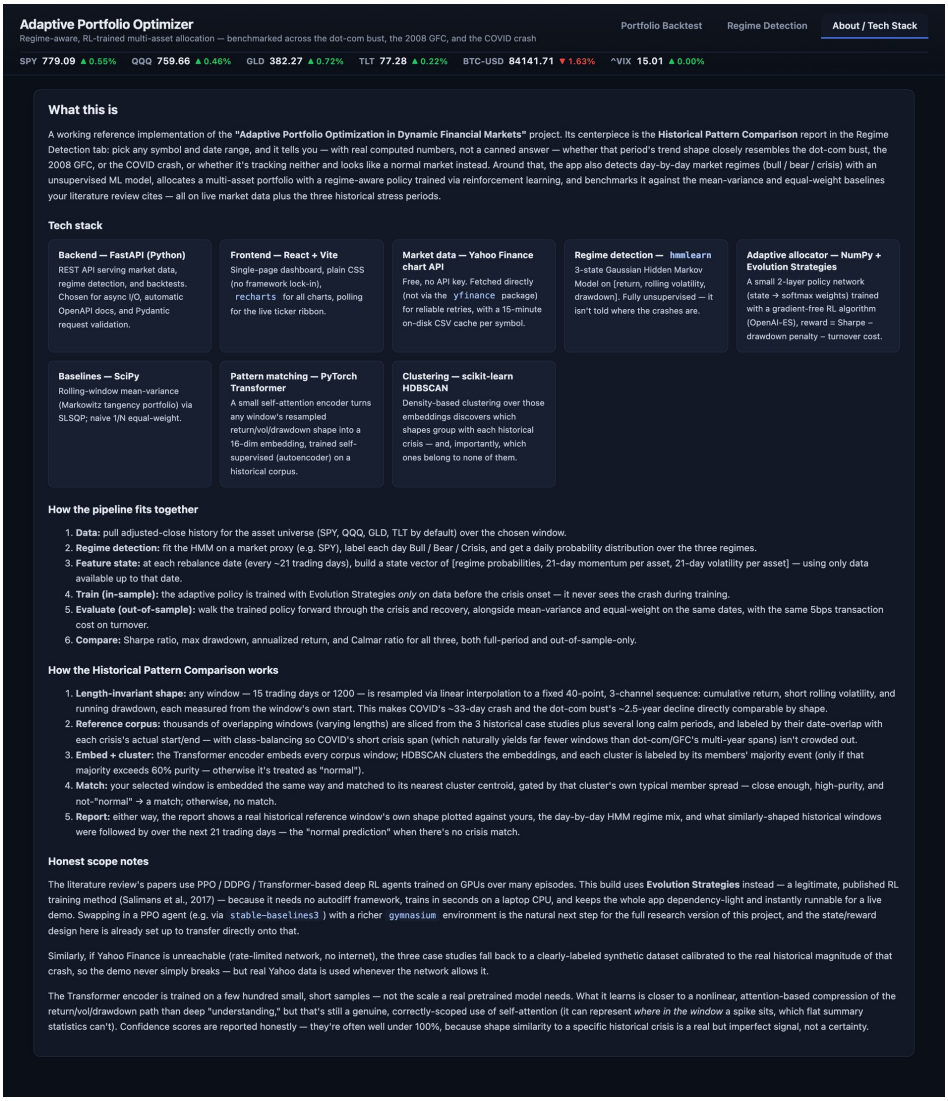


Figure 6.4 About and tech stack tab.

CHAPTER 7
RESULT AND DISCUSSION (COST ANALYSIS AS APPLICABLE)

7.1 Regime Detection

The unsupervised HMM marks the long turbulent stretches of all three crises as Crisis (Figure 7.1). The red band covers the 2008 and 2009 collapse, the low of October 2002 and the March 2020 crash, and the model was never told these dates. Crisis days are a minority of each window (19, 14 and 8 percent), and Bear states fill the slower phases before and after. In calm periods the labels flip between Bull and Bear more often, so we treat the HMM as a descriptive overlay and an input to the allocator, not as a trading signal.

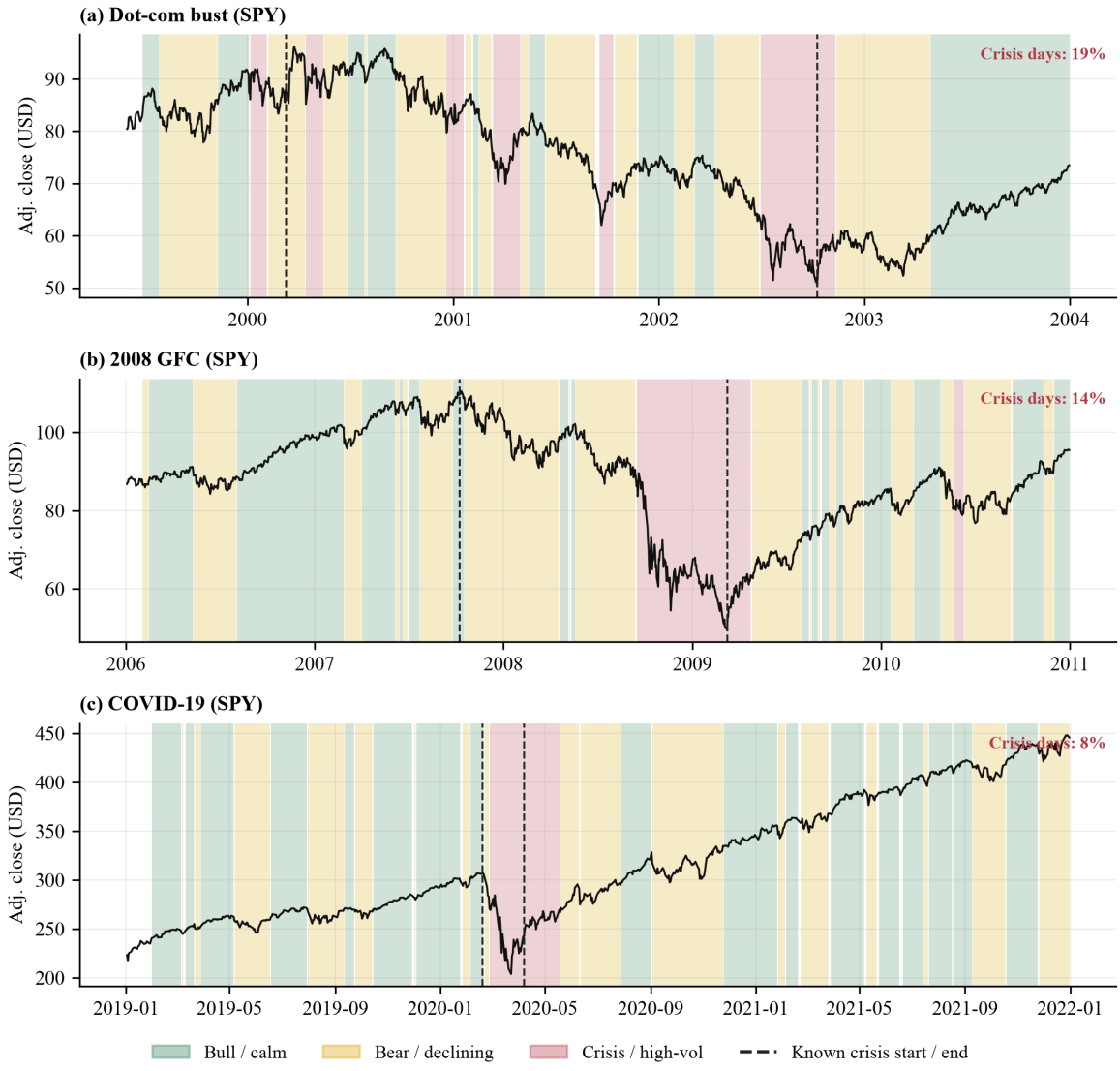


Figure 7.1 HMM regimes on SPY for (a) the dot-com bust, (b) the 2008 crisis and (c) the COVID-19 crash.

7.2 Historical-Pattern Comparison

7.2.1 Example Reports

Table 7.1 and Figure 7.2 show the report for five queries. The three crisis windows are matched to their own events with confidences of 59% (COVID-19), 72% (2008) and 89% (dot-com). The 2013 to 2015 period and the most recent months are labelled normal. These queries overlap the

periods that were used to build the corpus, so the matches are partly circular. They show what the report looks like, and the cross-validation below measures how well the method generalizes.

Table 7.1 Comparison report output for five example queries.

Query	Symbol and range	Days	Result (confidence)	Ann. return	Max DD	HMM bull / bear / crisis	Next-21d baseline
COVID-19 crash window	SPY 2020-01-15 to 2020-05-15	85	covid (59%)	-32.1%	-33.7%	32% / 38% / 29%	5.2%
2008 GFC window	SPY 2008-01-02 to 2009-06-30	377	gfc (72%)	-24.4%	-51.9%	57% / 28% / 15%	3.1%
Dot-com bust window	QQQ 2000-03-01 to 2002-10-01	649	dotcom (89%)	-46.0%	-82.4%	41% / 34% / 25%	-1.3%
Calm 2013-2015	SPY 2013-01-01 to 2015-12-31	756	normal (66%)	14.0%	-11.9%	57% / 37% / 6%	0.3%
Recent 2026 window	SPY 2026-03-01 to 2026-10-05	151	normal (77%)	23.5%	-7.7%	68% / 17% / 15%	0.9%

Crisis rows are scored in targeted mode against the matching event, the other two in auto mode. The last column is the mean 21-day forward return of the nearest same-label reference windows.

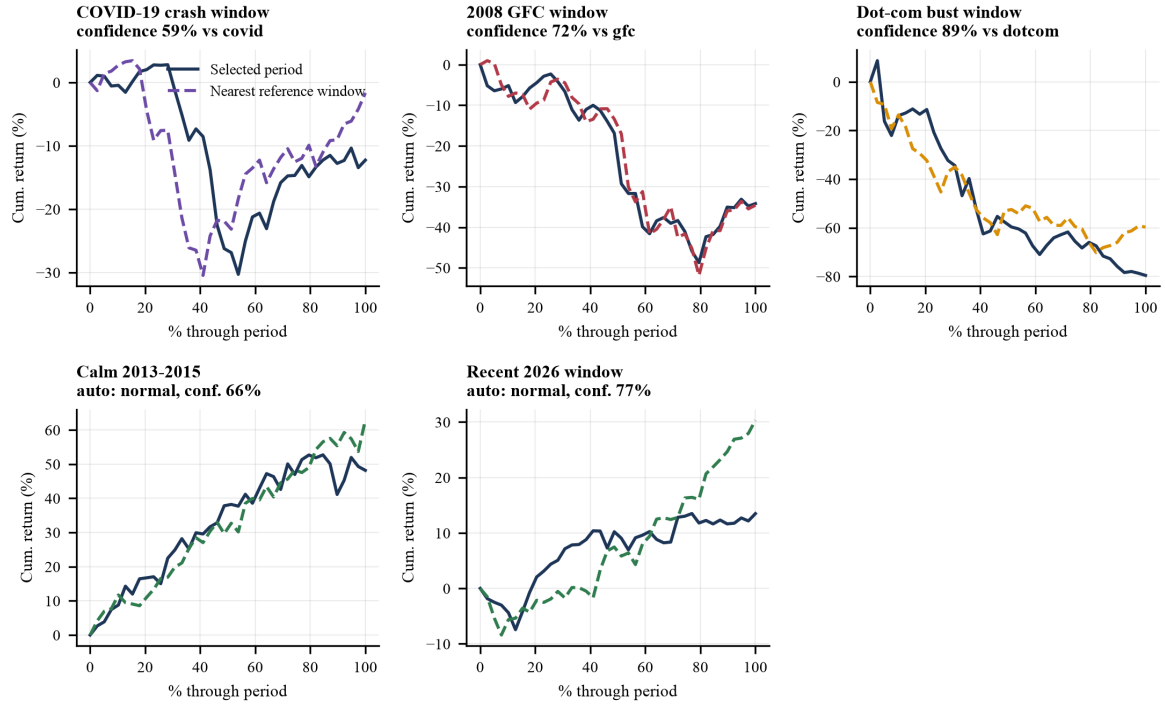


Figure 7.2 Query paths (solid) against the nearest reference windows (dashed).

7.2.2 Embedding Space

Figure 7.3 projects the 16-dimensional embeddings to two dimensions. Normal windows form the dense bulk. Event windows occupy a separate but overlapping region in which dot-com, 2008 and COVID-19 windows are mixed. That mixing is what we would expect if a crash followed by a recovery has a similar shape in every crisis.

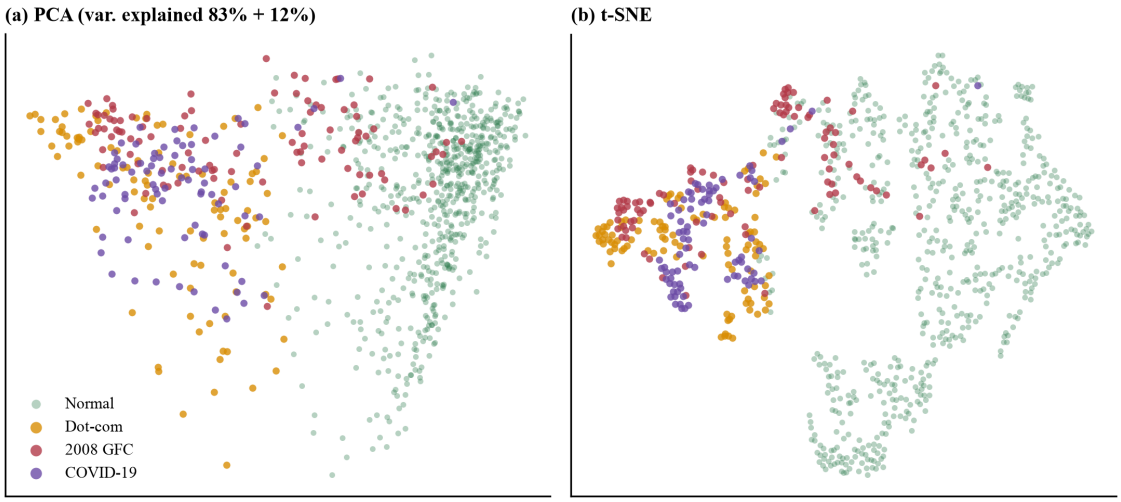


Figure 7.3 Corpus embeddings colored by label: (a) PCA and (b) t-SNE of the 965 vectors.

7.2.3 Leakage-Controlled Evaluation

Table 7.2 gives the blocked cross-validation. All variants reach about 75 to 76 percent accuracy, but almost all of it comes from the normal class, which has 99 percent recall. The trained Transformer reaches a balanced accuracy of 0.39 against a chance level of 0.25. Dot-com windows are recognized in 54% of cases, while 2008 and COVID-19 windows almost never are (recall 2% and 0%). The confusion matrix in Figure 7.5 shows that they are mostly assigned to dot-com or to normal. Telling crisis from normal works better: 49% of event windows are predicted as some crisis, with a false-alarm rate of 1.0%. So on unseen time blocks the representation captures "crisis-like" but not "which crisis".

Table 7.2 Blocked, purged 5-fold cross-validation of event identification.

Features (8-NN)	Balanced acc.	Macro-F1	Accuracy	Crisis recall	False alarm	Recall dot-com / 2008 / COVID / normal
Transformer (trained)	**0.389 ± 0.006**	**0.353 ± 0.005**	**75.8%**	**48.8%**	**1.0%**	**54% / 2% / 0% / 99%**
Transformer (untrained)	0.389 ± 0.009	0.355 ± 0.005	75.9%	46.6%	0.9%	54% / 2% / 0% / 99%
Raw shape (120-d)	0.367 ± 0.000	0.339 ± 0.000	75.0%	42.7%	0.7%	46% / 2% / 0% / 99%
PCA-16 of raw shape	0.372 ± 0.000	0.342 ± 0.000	75.1%	44.1%	0.9%	48% / 2% / 0% / 99%
Summary statistics (7-d)	0.378 ± 0.000	0.339 ± 0.000	75.2%	51.7%	1.2%	51% / 1% / 0% / 99%

Mean ± standard deviation over three encoder seeds for the two Transformer variants; the other variants are deterministic. Chance balanced accuracy is 0.25.

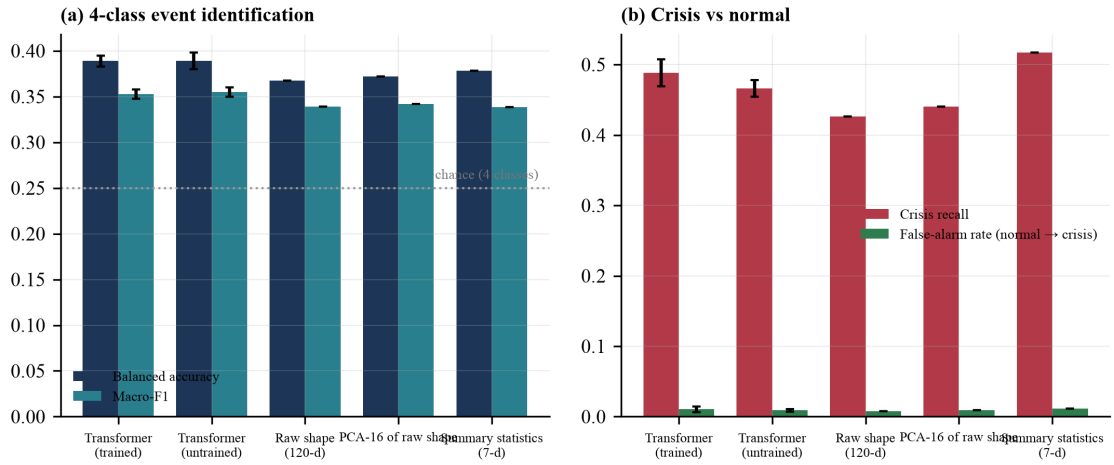


Figure 7.4 Embedding ablation under blocked cross-validation.

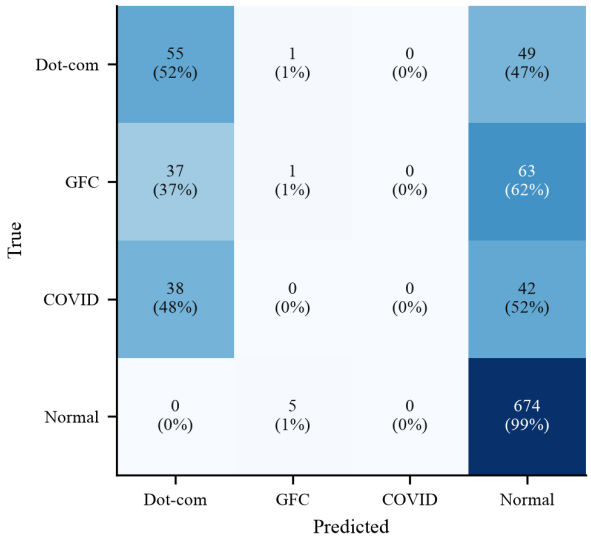


Figure 7.5 Confusion matrix of the trained-Transformer classifier under blocked cross-validation.

7.2.4 Unseen Crises

In leave-one-event-out tests (Table 7.3) the trained encoder flagged 90% of dot-com windows, 53% of 2008 windows and 93% of COVID-19 windows as crisis-like, although no window of that crisis was available for training. The 2008 crisis was a slow decline whose short sub-windows look like ordinary periods, which explains why it is the hardest. The result supports using the tool to ask whether a period looks like a crisis in general.

Table 7.3 Leave-one-event-out: share of the unseen crisis' windows predicted as crisis-like.

Held-out event	Test	Train	Transformer (trained)	Transformer (untrained)	Raw shape	PCA-16	Summary stats
Dot-com	105	808	90%	96%	85%	85%	90%
2008 crisis	101	799	53%	51%	51%	51%	53%
COVID-19	80	853	93%	82%	82%	88%	96%

7.3 Walk-Forward Allocation Backtests

Table 7.4 and Figure 7.6 to Figure 7.7 give the out-of-sample results. The picture is mixed and

depends heavily on the experimental design.

Dot-com as deployed. The adaptive portfolio reaches a Sharpe ratio of 0.84 and a maximum drawdown of -15.2%, ahead of both baselines. That result is not real. Its weights are 100 percent gold at all 40 rebalances and its turnover is zero. The gold futures series starts in August 2000, so the training segment is only 30 trading days and the policy collapses to a buy-and-hold gold position, which happened to rise.

Dot-com corrected (SPY, QQQ and VUSTX, 196 genuine pre-crisis training days). Now the adaptive policy loses badly, with a Sharpe ratio of -0.43 and a maximum drawdown of -73.2%. A policy trained in the bubble years concentrates in QQQ and then flips between QQQ and bonds (Figure 7.8).

2008 crisis. Mean-variance has the best Sharpe ratio (0.66). The adaptive policy has the lowest volatility (13.1%) but not the shallowest drawdown. **COVID-19.** Mean-variance leads again (1.15), and the adaptive policy (1.04) is close to equal-weight (1.03).

Table 7.4 Out-of-sample performance of the three strategies (policy seed 42).

Setting	Strategy	Total return	Ann. return	Ann. vol.	Sharpe	Max DD	Calmar	Turnover
Dot-com (as deployed)	Equal-Weight	-10.4%	-3.4%	20.9%	-0.16	-43.0%	-0.08	0.00
	Mean-Variance	52.6%	14.3%	20.3%	0.66	-17.7%	0.81	0.90
	Adaptive (ES)	**52.6%**	**14.3%**	**14.8%**	**0.84**	**-15.2%**	**0.94**	**0.00**
Dot-com (corrected)	Equal-Weight	-20.7%	-5.9%	21.2%	-0.28	-46.5%	-0.13	0.00
	Mean-Variance	**43.4%**	**10.0%**	**13.7%**	**0.61**	**-25.2%**	**0.39**	**0.50**
	Adaptive (ES)	-60.5%	-21.7%	41.6%	-0.43	-73.2%	-0.30	0.57
2008 GFC	Equal-Weight	28.8%	8.2%	14.7%	0.47	-26.6%	0.31	0.00
	Mean-Variance	**43.8%**	**11.9%**	**16.0%**	**0.66**	**-19.1%**	**0.62**	**0.75**
	Adaptive (ES)	20.2%	5.8%	13.1%	0.35	-23.8%	0.24	0.27
COVID-19	Equal-Weight	34.6%	17.2%	14.4%	1.03	-15.4%	1.11	0.00
	Mean-Variance	**40.1%**	**19.7%**	**14.9%**	**1.15**	**-12.4%**	**1.58**	**0.72**
	Adaptive (ES)	33.9%	16.8%	13.9%	1.04	-13.1%	1.28	0.72

The row with the highest Sharpe ratio in each setting is in bold. Turnover is the mean weight change per monthly rebalance; equal-weight targets never change, so it reports 0.

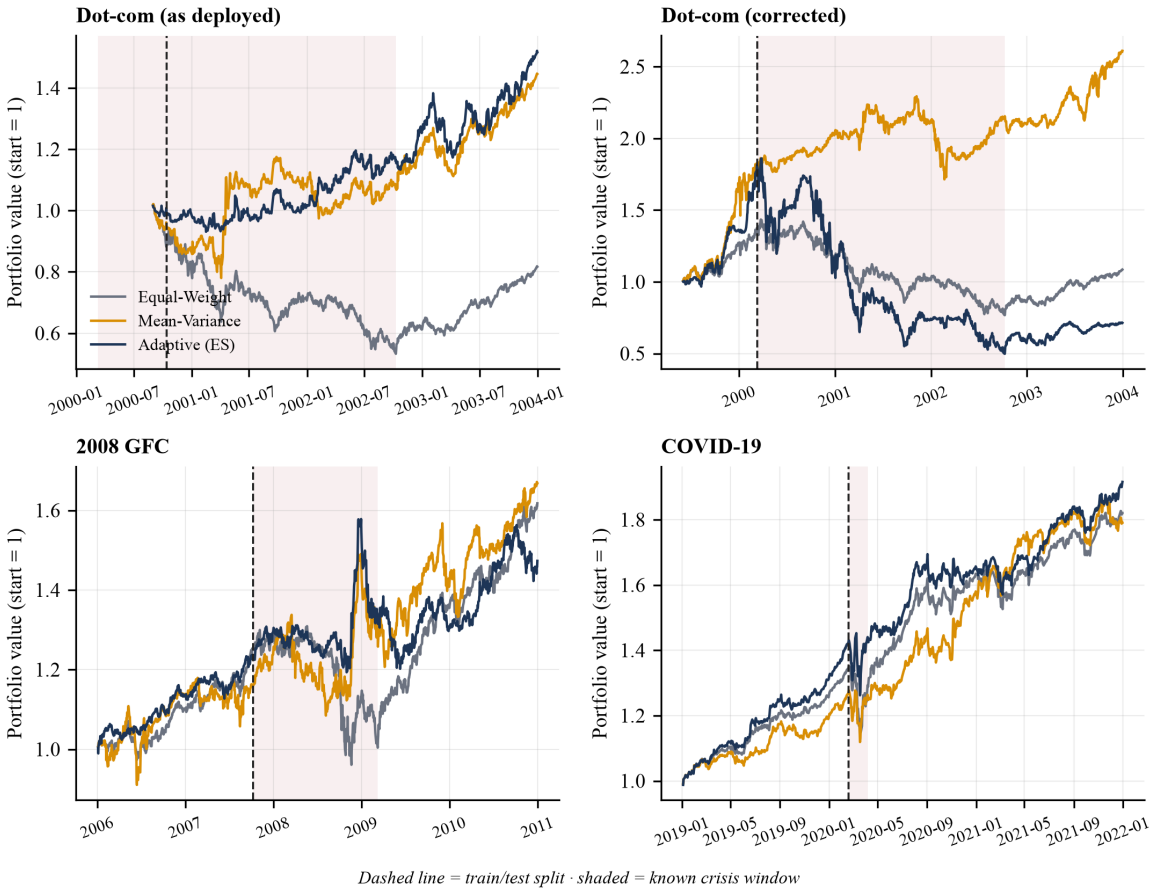


Figure 7.6 Portfolio value of the three strategies in the four settings.

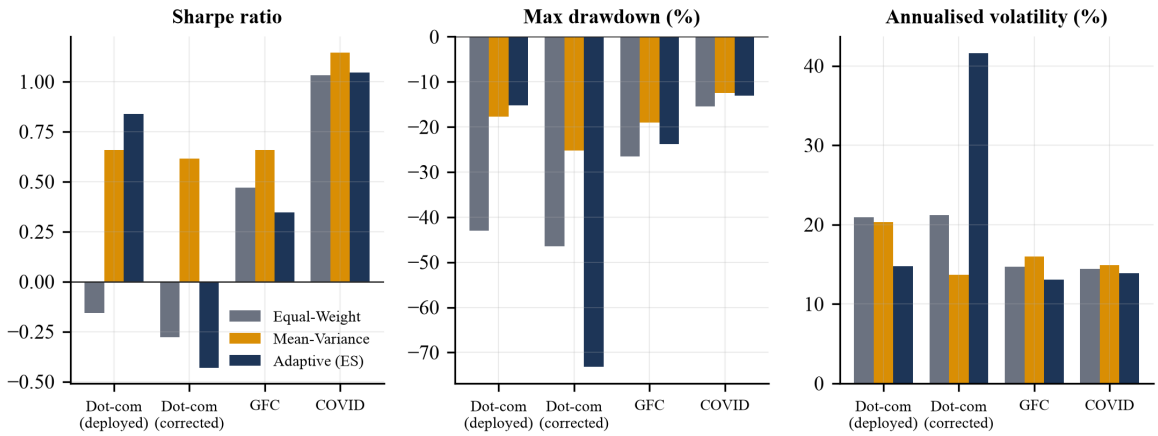


Figure 7.7 Out-of-sample Sharpe ratio, maximum drawdown and volatility.

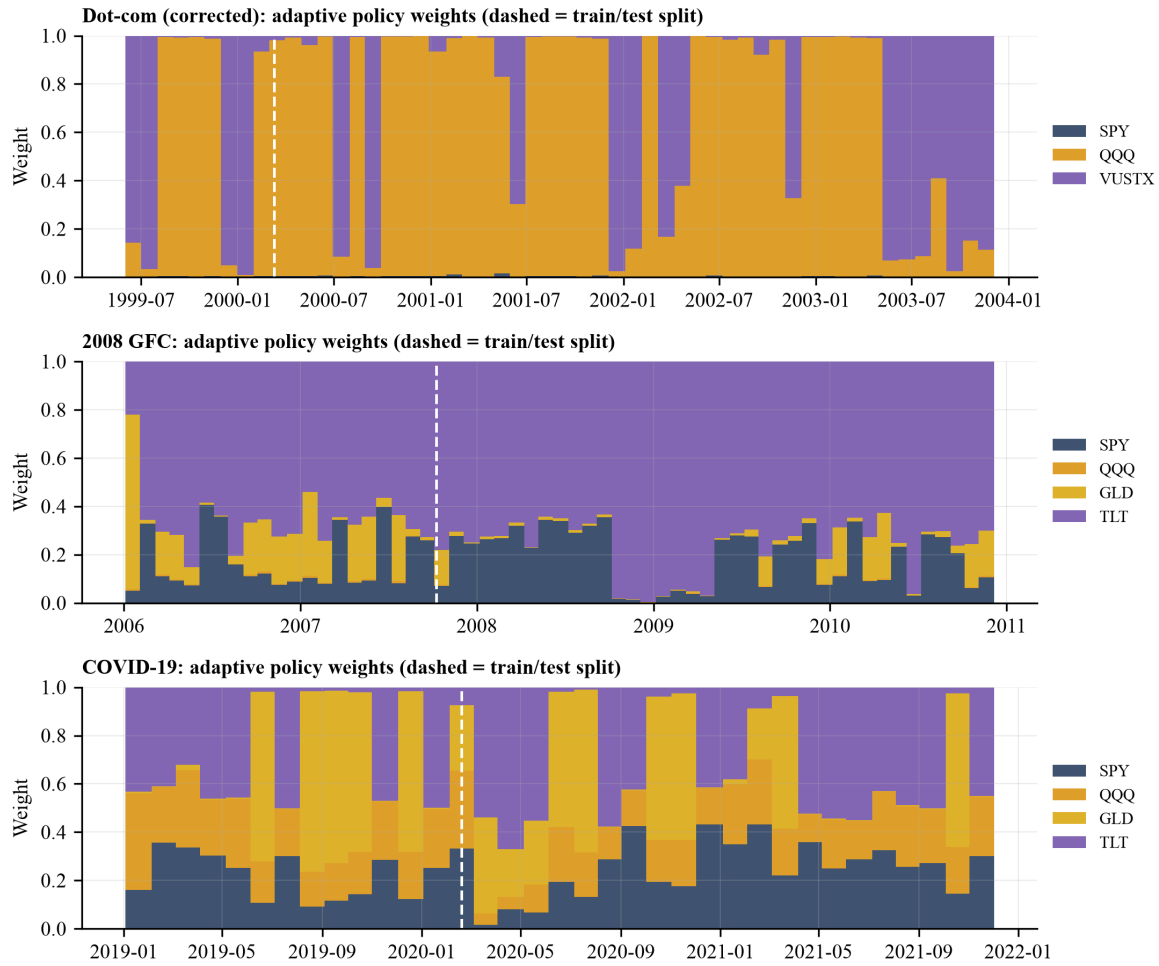


Figure 7.8 Adaptive-policy weights over time (seed 42).

7.3.1 Criterion-Wise Analysis

Table 7.5 Best strategy per criterion and setting on the test segment.

Criterion	Dot-com (as deployed)	Dot-com (corrected)	2008 GFC	COVID-19
Total return	AD	MV	MV	MV
Annualized volatility (lowest)	AD	MV	AD	AD
Sharpe ratio	AD	MV	MV	MV
Max drawdown (shallowest)	AD	MV	MV	MV
Calmar ratio	AD	MV	MV	MV

EW is Equal-Weight, MV is Mean-Variance, AD is Adaptive.

Mean-variance is best on return, Sharpe ratio, Calmar ratio and drawdown in three of the four settings. The adaptive policy has the lowest volatility in three settings, which is the quantity its reward penalizes through the drawdown and turnover terms, but Section 7.5 shows that this does not hold across seeds. Equal-weight is never best and has the deepest drawdown in three of four settings, yet it is the cheapest to run, with no optimization and no change in target weights.

7.4 Statistical Analysis

Table 7.6 Paired stationary-bootstrap intervals for the test-segment Sharpe ratio difference.

Setting	Adaptive minus	Sharpe difference	95% bootstrap CI	P(difference ≤ 0)
Dot-com (as deployed)	vs Mean-Variance	+0.17	[-0.77, +1.19]	0.36
	vs Equal-Weight	+0.91	[-0.46, +2.28]	0.10
Dot-com (corrected)	vs Mean-Variance	-1.04	[-2.27, +0.16]	0.95
	vs Equal-Weight	-0.15	[-0.53, +0.22]	0.79
2008 GFC	vs Mean-Variance	-0.31	[-1.45, +0.72]	0.72
	vs Equal-Weight	-0.13	[-1.47, +1.15]	0.60
COVID-19	vs Mean-Variance	-0.10	[-0.94, +0.77]	0.59
	vs Equal-Weight	+0.02	[-0.84, +0.79]	0.51

Every interval contains zero. With test segments of 473 to 957 daily returns, none of the observed Sharpe ratio differences can be told apart from zero in either direction, so neither the dot-com "win" nor the other "losses" should be read as evidence about the method. Differences between seeds are a larger effect than most of the differences between strategies (Section 7.5). The standard deviation of balanced accuracy across encoder seeds is at most 0.009, so the matcher results are stable even though the allocator results are not. We report eight comparisons without a multiplicity correction. Since no interval excludes zero, a correction would not change any conclusion.

7.5 Ablation Study

Table 7.7 and Figure 7.9 give the mean and standard deviation of the test-segment results over six seeds. Removing the regime features does not change the outcome in any consistent direction. The 2008 Sharpe ratio drops slightly and the COVID-19 one rises, both within one standard deviation. A more telling result is that the untrained random policy matches or beats the trained one in the 2008 and COVID-19 settings (Sharpe 0.59 against 0.36 and 1.02 against 0.90), and has lower volatility in COVID-19 (14.1 against 19.2 percent). Training by evolution strategies on the short pre-crisis segment overfits and does not carry over to the crisis. In the as-deployed dot-com setting the trained variants give identical results for every seed because they all converge to the same constant gold portfolio.

Table 7.7 Allocator ablation: test-segment results, mean ± standard deviation over six seeds.

Setting	Strategy or variant	Sharpe	Ann. volatility	Max drawdown	Seeds with lowest volatility
Dot-com (as deployed)	Equal-Weight	-0.16	20.9%	-43.0%	n/a
	Mean-Variance	0.66	20.3%	-17.7%	n/a
	Adaptive (full)	0.84 ± 0.00	14.8% ± 0.0%	-15.2% ± 0.0%	100%
	Adaptive (no regime)	0.84 ± 0.00	14.8% ± 0.0%	-15.2% ± 0.0%	100%
	Adaptive (untrained)	-0.27 ± 0.03	26.9% ± 2.7%	-54.3% ± 4.4%	0%
Dot-com (corrected)	Equal-Weight	-0.28	21.2%	-46.5%	n/a
	Mean-Variance	0.61	13.7%	-25.2%	n/a
	Adaptive (full)	-0.42 ± 0.08	41.3% ± 3.0%	-76.5% ± 5.9%	0%
	Adaptive (no regime)	-0.45 ± 0.06	41.4% ± 2.8%	-77.5% ± 4.8%	0%
	Adaptive (untrained)	-0.37 ± 0.05	27.5% ± 3.4%	-59.8% ± 6.3%	0%
2008 GFC	Equal-Weight	0.47	14.7%	-26.6%	n/a
	Mean-Variance	0.66	16.0%	-19.1%	n/a
	Adaptive (full)	0.36 ± 0.23	13.9% ± 3.5%	-25.1% ± 8.6%	67%
	Adaptive (no regime)	0.30 ± 0.19	15.9% ± 3.7%	-29.7% ± 8.7%	33%
	Adaptive (untrained)	0.59 ± 0.16	14.1% ± 0.9%	-24.1% ± 0.4%	50%
COVID-19	Equal-Weight	1.03	14.4%	-15.4%	n/a
	Mean-Variance	1.15	14.9%	-12.4%	n/a
	Adaptive (full)	0.90 ± 0.12	19.2% ± 3.5%	-21.5% ± 7.1%	0%
	Adaptive (no regime)	0.97 ± 0.13	19.6% ± 2.5%	-21.4% ± 5.1%	0%
	Adaptive (untrained)	1.02 ± 0.05	14.1% ± 0.8%	-14.0% ± 0.1%	33%

Baselines are deterministic. The last column is the share of seeds in which the adaptive variant has lower volatility than both baselines.

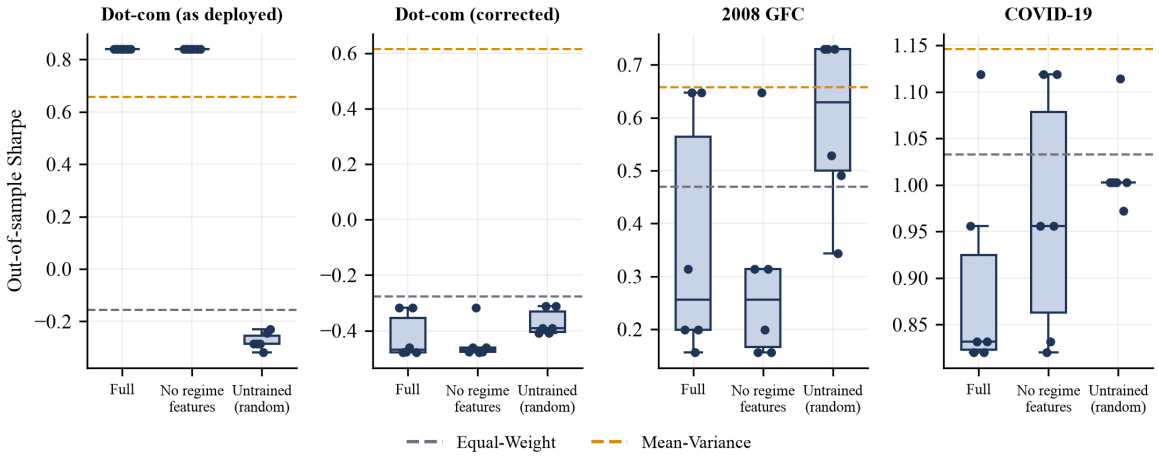


Figure 7.9 Out-of-sample Sharpe ratio of the adaptive policy across six seeds; dashed lines are the baselines.

For the matcher, replacing the trained Transformer with an untrained one, the raw shape, PCA or

seven summary statistics changes balanced accuracy by at most 0.022 (Table 7.2). The leave-one-event-out rates (Table 7.3) are similar: summary statistics are highest for COVID-19 (96 against 93 percent for the trained encoder), the untrained encoder is highest for dot-com (96 against 90 percent), and the trained encoder is above the raw features for dot-com and COVID-19. The useful signal sits in the resampled shape itself, and at this data scale the Transformer neither adds to it nor removes any of it.

7.6 Comparison with Existing Methods

A number-for-number comparison with the four reviewed studies is not possible. They use different data, assets, periods and costs, and their code and data were not available to us. We therefore compare against equal-weight and mean-variance under one protocol, and compare capabilities with the reviewed approaches in Table 7.8, using only the properties that those studies report.

Table 7.8 Capability comparison with the reviewed studies.

Capability	Mean-Variance / 1/N	Zhang (GAT)	DRL-APO	RL-APM	Regime-aware RL	This work
Multi-asset allocation	Yes	Yes	Yes	No (single stock)	Yes	Yes (3 to 4 assets)
Explicit regime detection	No	Not reported	No	No	Yes	Yes (HMM)
Transaction costs in objective	No	Not reported	Not reported	Ignored	Not reported	Yes (turnover and 5 bps)
Historical-analogue comparison	No	No	No	No	No	Yes (Transformer and HDBSCAN)
Interpretable output	Yes	Low	Low	Medium	Medium (LSTM)	Yes (report and overlay)
Leakage-controlled, multi-seed evaluation	n/a	Not reported	Not reported	Not reported	Not reported	Yes
Reported to beat static baselines	n/a	Yes (reported)	Yes (reported)	Yes (reported)	Yes (reported)	No (this report)

"Not reported" means the literature review did not record the property, not that the study lacks it.

The difference lies in scope and in how the results are reported. The application adds an explainable comparison and an evaluation with seeds, ablations and leakage controls. It does not show that its allocator beats the baselines.

7.7 Cost Analysis

The monetary cost of the project was nil (Section 3.2.2). Table 7.9 gives the computing cost on the development laptop. Training and scoring are cheap because the Transformer and the corpus are small.

Table 7.9 Computing cost of the main operations.

Operation	Time on Apple M3 (CPU)
Build the matcher (train encoder, embed 965 windows, HDBSCAN grid), once per server start	about 25 s
One comparison request after the matcher exists	about 0.1 s
One backtest including HMM fit and 40 ES generations	8 to 17 s

The trading cost inside the backtests is modelled at 5 basis points of turnover. Mean-Variance trades heavily (0.75 per rebalance in 2008, 0.72 in COVID-19), and pays for it in return relative to a frictionless run. TC15 in Table 5.5 confirms that a higher cost lowers net return.

7.8 Discussion

- **Regimes can be found without labels, but the label is not a forecast.** The HMM recovers the turbulent phases of all three crises. Its day-level states are noisy, and removing them from the allocator's input made no consistent difference.
- **Shape similarity detects crises and cannot yet name them.** Once windows are reduced to normalized shapes, the three crises look alike, so telling them apart on held-out blocks fails while crisis against normal works with few false alarms. The confidence in the report is closeness to a family of crash shapes. It is not the probability that the period is that event.
- **The Transformer is not where the signal comes from.** An untrained encoder, the raw shape and seven summary statistics do as well. With under a thousand strongly overlapping windows this is expected, and the encoder acts close to a fixed random projection that preserves distances.
- **The adaptive allocator did not beat simple baselines.** Mean-variance is the strongest or near strongest in three of four settings, and apparent wins were fragile.
- **The experimental design can decide the conclusion.** Swapping gold futures for a bond fund in the dot-com setting turned the best strategy into the worst.

7.9 Limitations

- Three crises and four calm periods give 965 overlapping windows with a large normal class. Blocked purging reduces the dependence between windows but cannot remove it, and other crises (1987, 1998, 2022) were not tested.
- The example queries in Table 7.1 overlap the corpus; only the blocked and leave-one-event-out results measure generalization.
- Evolution strategies replace the PPO and DDPG agents of the literature, with a small state, 40 generations and short training segments. The negative result applies to this implementation and says little about deep reinforcement learning in general. Only long-only portfolios of three or four assets were studied, with one cost assumption and no liquidity or market-impact model, which was one of the original objectives.
- The as-deployed dot-com setting is invalid and is kept only to document the diagnosis. The corrected setting uses a mutual fund as a proxy for bonds, and each test segment contains a

single crisis.

- Results move slightly when Yahoo refreshes its data, as the COVID-19 case showed. The endpoint is unofficial and may change, and adjusted prices are revised retroactively.

CHAPTER 8
CONCLUSION

We built and evaluated a web application that compares any market period with the dot-com bust, the 2008 crisis and the COVID-19 crash. It combines a length-invariant shape representation, a Transformer encoder and nearest-neighbor scoring over an HDBSCAN-structured corpus with an HMM regime view and a walk-forward allocation workbench. The evaluation gives a clear but modest result. The comparison separates crisis-like from normal periods (crisis recall 49% at a 1.0% false-alarm rate) and recognizes unseen crises as crisis-like. It cannot say which crisis a period resembles, and neither the trained Transformer nor the regime-aware allocator beat simpler alternatives.

Table 8.1 compares the outcome with the objectives from Section 2.3.

Table 8.1 Objectives and outcomes.

Objective	Outcome	Status
1. Adaptive allocation model for multi-asset portfolios	Implemented as an evolution-strategies policy rather than PPO or DDPG; works on 3 to 4 assets	Partly met
2. Real-time regime detection to trigger reallocation	HMM regimes feed the policy state; ablation shows no consistent benefit	Met, benefit not shown
3. Transaction costs and liquidity in the reward	Turnover cost and drawdown in the reward; no liquidity model	Partly met
4. Benchmark against mean-variance and equal-weight	Sharpe, drawdown, return, volatility and Calmar reported, with bootstrap intervals	Met
5. Robustness across bull, bear and volatile regimes	Four settings, six seeds and ablations; no consistent advantage found	Met, result negative
6. Historical comparison with Transformer and HDBSCAN	Working report with overlay, statistics and baseline; evaluated with leakage controls	Met
7. Working web application on live data	Eight endpoints, three-tab dashboard, 21 of 21 tests passed	Met

We regard the diagnostics as useful results in their own right: the invalid split that produced a constant gold portfolio, the untrained-encoder and untrained-policy controls, and the seed and bootstrap analysis.

8.1 Future Work

- Extend the corpus to more crises and markets, and cross-validate by whole event instead of by window.
- Replace the single-crisis labels by a continuous severity score, and add shape descriptors that keep the original duration so that fast and slow crashes can be told apart.
- Pre-train the encoder on a much larger set of unlabelled windows, or try contrastive objectives, before claiming any benefit from attention.
- Implement PPO and DDPG agents under the same protocol with longer training windows, rolling retraining, policy regularization and a cost and liquidity model.

- Add automated tests to the repository and make the dot-com configuration valid by default.
- Move toward TRL 4 and 5 with paper trading on live data, and evaluate the dashboard with real users.

CHAPTER 9
REFERENCES

[1] R. Cont, "Empirical properties of asset returns: Stylized facts and statistical issues," *Quant. Finance*, vol. 1, no. 2, pp. 223-236, 2001.

[2] H. Markowitz, "Portfolio selection," *J. Finance*, vol. 7, no. 1, pp. 77-91, 1952.

[3] V. DeMiguel, L. Garlappi, and R. Uppal, "Optimal versus naive diversification: How inefficient is the 1/N portfolio strategy?," *Rev. Financ. Stud.*, vol. 22, no. 5, pp. 1915-1953, 2009.

[4] J. D. Hamilton, "A new approach to the economic analysis of nonstationary time series and the business cycle," *Econometrica*, vol. 57, no. 2, pp. 357-384, 1989.

[5] A. Ang and G. Bekaert, "International asset allocation with regime shifts," *Rev. Financ. Stud.*, vol. 15, no. 4, pp. 1137-1187, 2002.

[6] J. Moody and M. Saffell, "Learning to trade via direct reinforcement," *IEEE Trans. Neural Netw.*, vol. 12, no. 4, pp. 875-889, Jul. 2001.

[7] Z. Jiang, D. Xu, and J. Liang, "A deep reinforcement learning framework for the financial portfolio management problem," arXiv:1706.10059, 2017.

[8] B. Zhang, "Graph attention-based heterogeneous multi-agent deep reinforcement learning for adaptive portfolio optimization," *Sci. Rep.*, vol. 15, 2025.

[9] "Deep reinforcement learning for adaptive portfolio optimization in dynamic financial environments," *Appl. Artif. Intell. Comput. Syst.*, 2025.

[10] "Reinforcement learning for single-stock portfolio optimization in equity markets," *Int. J. Comput. Intell. Syst.*, Springer, 2025.

[11] "Adaptive and regime-aware RL for portfolio optimization," arXiv:2509.14385, 2025.

[12] W. F. Sharpe, "The Sharpe ratio," *J. Portf. Manag.*, vol. 21, no. 1, pp. 49-58, 1994.

[13] L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proc. IEEE*, vol. 77, no. 2, pp. 257-286, 1989.

[14] "hmmlearn: Hidden Markov models in Python, with scikit-learn like API." [Online]. Available: https://github.com/hmmlearn/hmmlearn. Accessed: Oct. 2026.

[15] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2016.

[16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[17] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," arXiv:1703.03864, 2017.

[18] A. Vaswani *et al.*, "Attention is all you need," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998-6008.

[19] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, "A transformer-based framework for multivariate time series representation learning," in *Proc. 27th ACM*

SIGKDD Conf. Knowl. Discov. Data Min., 2021, pp. 2114-2124.

[20] R. J. G. B. Campello, D. Moulavi, and J. Sander, "Density-based clustering based on hierarchical density estimates," in *Adv. Knowl. Discov. Data Min. (PAKDD)*, LNCS 7819, 2013, pp. 160-172.

[21] L. McInnes, J. Healy, and S. Astels, "hdbscan: Hierarchical density based clustering," *J. Open Source Softw.*, vol. 2, no. 11, p. 205, 2017.

[22] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011.

[23] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," Resolution A/RES/70/1, 2015.

[24] J. C. Mankins, "Technology readiness levels: A white paper," NASA Office of Space Access and Technology, 1995.

[25] Yahoo Finance, "Historical market data (chart API)." [Online]. Available: https://finance.yahoo.com. Accessed: Oct. 2026.

[26] D. Hendrycks and K. Gimpel, "Gaussian error linear units (GELUs)," arXiv:1606.08415, 2016.

[27] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," arXiv:1607.06450, 2016.

[28] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[29] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8024-8035.

[30] D. N. Politis and J. P. Romano, "The stationary bootstrap," *J. Am. Stat. Assoc.*, vol. 89, no. 428, pp. 1303-1313, 1994.

APPENDIX A: SAMPLE CODE

A.1 Transformer shape encoder (ml/embedding.py)

```
class ShapeEncoder(nn.Module):
    """(batch, 40, 3) -> (batch, 16) embedding via a small Transformer encoder."""

    def __init__(self):
        super().__init__()
        self.input_proj = nn.Linear(3, D_MODEL)
        self.pos_embed = nn.Parameter(torch.randn(1, WINDOW_POINTS, D_MODEL) * 0.02)
        layer = nn.TransformerEncoderLayer(
            d_model=D_MODEL, nhead=N_HEADS, dim_feedforward=D_MODEL * 2,
            dropout=0.0, batch_first=True, activation="gelu",
        )
        self.encoder = nn.TransformerEncoder(layer, num_layers=N_LAYERS)
        self.out_proj = nn.Linear(D_MODEL, EMBED_DIM)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        h = self.input_proj(x) + self.pos_embed
        h = self.encoder(h)
        pooled = h.mean(dim=1)
        return self.out_proj(pooled)
```

A.2 Length-invariant shape (ml/embedding.py)

```
def build_channels(prices: pd.Series) -> np.ndarray:
    """
    In-window-only features -- no external lookback required, so this is
    safe to call on corpus slices as short as ~15 trading days (unlike
    regime.py's _build_features, which needs 20-60 days of PRIOR context).

    Returns an (n_days, 3) array: [cumulative return from window start,
    short rolling annualized vol, running drawdown from window start].
    """
    prices = prices.astype(float)
    cum_return = (prices / prices.iloc[0] - 1.0).values

    daily_ret = prices.pct_change().fillna(0.0)
    roll_vol = daily_ret.rolling(SHORT_VOL_WINDOW, min_periods=1).std().fillna(0.0)
        * np.sqrt(252)
    roll_vol = roll_vol.values

    running_max = prices.cummax()
    drawdown = (prices / running_max - 1.0).values

    return np.column_stack([cum_return, roll_vol, drawdown]).astype(np.float32)
```

```
def resample_to_fixed(channels: np.ndarray, n_points: int = WINDOW_POINTS) ->
    np.ndarray:
    """Per-channel linear interpolation onto n_points along a normalized [0,1]
        x-axis."""
    n_days = channels.shape[0]
    if n_days == 1:
        return np.repeat(channels, n_points, axis=0)
    src_x = np.linspace(0.0, 1.0, n_days)
    dst_x = np.linspace(0.0, 1.0, n_points)
    out = np.empty((n_points, channels.shape[1]), dtype=np.float32)
    for c in range(channels.shape[1]):
        out[:, c] = np.interp(dst_x, src_x, channels[:, c])
    return out
```

A.3 Label scoring (ml/event_matcher.py)

```
def _score_label(query_emb: np.ndarray, matcher: "Matcher", label: str) -> dict:
    """
    Score how strongly this query resembles `label`, using the actual
    labelled corpus windows rather than any cluster summary -- so HDBSCAN
    noise points (~16% of the corpus, belonging to no cluster at all) still
    contribute, instead of being invisible to the score.
```

```
    match_dist is the mean distance to the MATCH_K nearest windows CARRYING
    that label; it's expressed in units of that label's own in-class
    spacing (_label_scales), then passed through a Gaussian kernel. That
    calibrates the way a reader expects: a query sitting well inside the
    spacing genuine members keep from each other reads ~0.9, one sitting at
    exactly the typical spacing reads ~0.6, and one 2-3x outside decays to
    ~0.1 and below.
    """
    dists = np.linalg.norm(matcher.embeddings - query_emb, axis=1)
    same_label_idxs = np.array([i for i, w in enumerate(matcher.corpus) if w.label
        == label])
    k = min(MATCH_K, len(same_label_idxs))
    nearest_same = same_label_idxs[np.argsort(dists[same_label_idxs])[:k]]

    match_dist = float(dists[nearest_same].mean())
    scale = matcher.label_scales[label]
    proximity = float(np.exp(-0.5 * (match_dist / scale) ** 2))
    purity = _local_purity(query_emb, matcher, label)

    fwd = [matcher.corpus[i].forward_return_21d for i in nearest_same
           if matcher.corpus[i].forward_return_21d is not None]
    return {
        "label": label,
        "confidence": proximity * purity,
        "purity": purity,
        "match_dist": match_dist,
        "typical_dist": scale,
        "nearest_idx": int(nearest_same[0]),
        "neighbour_idxs": nearest_same,
        "forward_return_21d_mean": float(np.mean(fwd)) if fwd else None,
        "forward_return_21d_std": float(np.std(fwd)) if fwd else None,
        "forward_return_21d_n": len(fwd),
    }
```

A.4 Mean-variance weights and adaptive policy (ml/allocator.py)

```
def mean_variance_weights(returns_window: pd.DataFrame, risk_free_daily: float =
    0.0) -> np.ndarray:
    """
    Max-Sharpe (tangency) portfolio over a trailing window of daily returns,
    long-only, fully invested. Falls back to equal-weight if the optimizer
    fails or the window is degenerate (e.g. all-zero variance).
    """
    n = returns_window.shape[1]
    mean_ret = returns_window.mean().values
    cov = returns_window.cov().values
    cov += np.eye(n) * 1e-6  # numerical stability

    def neg_sharpe(w):
        port_ret = w @ mean_ret
        port_vol = np.sqrt(w @ cov @ w)
        if port_vol < 1e-9:
            return 0.0
        return -(port_ret - risk_free_daily) / port_vol

    cons = ({"type": "eq", "fun": lambda w: np.sum(w) - 1.0},)
    bounds = tuple((0.0, 1.0) for _ in range(n))
    x0 = equal_weights(n)
    result = minimize(neg_sharpe, x0, method="SLSQP", bounds=bounds, constraints=cons,
                      options={"maxiter": 200, "ftol": 1e-9})
    if not result.success or np.any(np.isnan(result.x)):
        return x0
    w = np.clip(result.x, 0, None)
    total = w.sum()
    return w / total if total > 1e-9 else x0
```

```
class PolicyNet:
    """Minimal 2-layer MLP: state -> tanh(hidden) -> softmax(weights). Pure numpy."""
    w1: np.ndarray
    b1: np.ndarray
    w2: np.ndarray
    b2: np.ndarray
```

```
    @classmethod
    def random_init(cls, state_dim: int, n_assets: int, hidden: int = 16, rng=None)
        -> "PolicyNet":
        rng = rng or np.random.default_rng()
        scale1 = np.sqrt(2.0 / state_dim)
        scale2 = np.sqrt(2.0 / hidden)
        return cls(
            w1=rng.normal(0, scale1, size=(state_dim, hidden)),
            b1=np.zeros(hidden),
            w2=rng.normal(0, scale2, size=(hidden, n_assets)),
            b2=np.zeros(n_assets),
        )

    def flatten(self) -> np.ndarray:
        return np.concatenate([self.w1.ravel(), self.b1.ravel(), self.w2.ravel(),
            self.b2.ravel()])

    def shapes(self):
        return self.w1.shape, self.b1.shape, self.w2.shape, self.b2.shape

    @classmethod
    def from_flat(cls, flat: np.ndarray, state_dim: int, n_assets: int, hidden: int
        = 16) -> "PolicyNet":
        i = 0
        w1 = flat[i:i + state_dim * hidden].reshape(state_dim, hidden); i +=
            state_dim * hidden
        b1 = flat[i:i + hidden]; i += hidden
        w2 = flat[i:i + hidden * n_assets].reshape(hidden, n_assets); i += hidden *
            n_assets
        b2 = flat[i:i + n_assets]
        return cls(w1, b1, w2, b2)

    def act(self, state: np.ndarray) -> np.ndarray:
        h = np.tanh(state @ self.w1 + self.b1)
        logits = h @ self.w2 + self.b2
        logits = logits - logits.max()
        exp = np.exp(logits)
        return exp / exp.sum()
```

A.5 Evolution strategies training loop (ml/allocator.py)

```
def train_adaptive_policy(returns: pd.DataFrame, regime_proba: pd.DataFrame,
    rebalance_idxs: list[int],
                          cost_bps: float = 5.0, hidden: int = 16, generations: int
                              = 40,
                          population: int = 32, sigma: float = 0.1, lr: float = 0.05,
                          seed: int = 42) -> PolicyNet:
    """
    Train the adaptive allocation policy with a standard OpenAI-ES update:
    perturb the parameter vector with antithetic Gaussian noise, weight each
    perturbation by its (rank-normalized) reward, and step in that direction.
    Runs entirely on the TRAIN segment of the backtest window (see engine.py)
    so evaluation on the TEST segment is genuinely out-of-sample.
    """
    rng = np.random.default_rng(seed)
    n_assets = returns.shape[1]
    state_dim = 3 + n_assets * 2  # regime probs + momentum + vol per asset
    theta = PolicyNet.random_init(state_dim, n_assets, hidden, rng).flatten()

    if len(rebalance_idxs) < 2:
        return PolicyNet.from_flat(theta, state_dim, n_assets, hidden)

    for gen in range(generations):
        noise = rng.normal(0, 1, size=(population // 2, theta.size))
        noise = np.concatenate([noise, -noise], axis=0)  # antithetic sampling
        rewards = np.empty(noise.shape[0])
        for i in range(noise.shape[0]):
            candidate = theta + sigma * noise[i]
            rewards[i] = _simulate_policy(candidate, state_dim, n_assets, hidden,
                                          returns, regime_proba, rebalance_idxs,
                                              cost_bps)["reward"]
        ranks = rewards.argsort().argsort().astype(float)
        std_reward = (ranks - ranks.mean()) / (ranks.std() + 1e-9)
        theta = theta + (lr / (noise.shape[0] * sigma)) * (noise.T @ std_reward)
```

```
    return PolicyNet.from_flat(theta, state_dim, n_assets, hidden)
```

A.6 Comparison route (main.py)

```
def regimes_compare(
    symbol: str = "SPY", start: str | None = None, end: str | None = None,
    event_id: str | None = None,
):
    if not start or not end:
        raise HTTPException(status_code=400, detail="start and end are required")
    try:
        return event_matcher.match_window(symbol, start, end, target_event=event_id)
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    except md.MarketDataError as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc
    except Exception as exc:
        logger.exception("regimes_compare failed")
        raise HTTPException(status_code=500, detail=str(exc)) from exc
```

A.7 Commands to reproduce the experiments

```
./run.sh                                                # API and dashboard
backend/.venv/bin/python paper/scripts/exp_backtest.py   # backtests, bootstrap
backend/.venv/bin/python paper/scripts/exp_ablation.py   # allocator ablation
backend/.venv/bin/python paper/scripts/exp_matcher.py    # matcher CV and LOEO
backend/.venv/bin/python paper/scripts/run_tests.py      # 21 test cases
```